

Tarea Corta #1: Observability

Equipo de trabajo: Granados Retana, Diego - 2022158363 Granados Retana, Daniel - 2022104692 Mora Montes, Diego - 2022104866 Karolyi Gutierrez, Gunther - 2017238873 Gutierrez Conejo, Eduardo - 2019073558

Guía de instalación y uso

Prerequisitos

Para instalar Gatling, se necesitan varias aplicaciones. Primero se necesita un editor de texto que esté configurado para correr Java, como IntelliJ o NetBeans. Se necesita instalar Maven para descargar las dependencias. Se recomienda tener también el JDK más actualizado, pero puede funcionar sin él. Luego, se tienen que importar las librerías de Gatling. Esto se hace poniendo las dependencias en el pom.xml del proyecto para que luego Maven las instale.

```
<dependencies>
  <dependency>
    <groupId>io.gatling.highcharts</groupId>
    <artifactId>gatling-charts-highcharts</artifactId>
    <version>${gatling.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>io.gatling</groupId>
    <artifactId>gatling-app</artifactId>
    <version>3.9.5</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<plugin>
  <groupId>io.gatling</groupId>
  <artifactId>gatling-maven-plugin</artifactId>
  <version>${gatling-maven-plugin.version}</version>
  <configuration>
    <!-- Enterprise Cloud (https://cloud.gatling.io/) configuration
reference:
https://gatling.io/docs/gatling/reference/current/extensions/maven_plugin/#working
-with-gatling-enterprise-cloud -->
    <!-- Enterprise Self-Hosted configuration reference:
https://gatling.io/docs/gatling/reference/current/extensions/maven_plugin/#working
-with-gatling-enterprise-self-hosted -->
  </configuration>
</plugin>
```

Para utilizar Gatling, es recomendado descargar un proyecto base y adaptarlo para nuestro proyecto. Uno se puede descargar de aquí: <https://github.com/gatling/gatling-maven-plugin-demo-java/tree/main>

Este proyecto contiene lo necesario para correr Gatling. Para crear nuestra propia simulación o prueba de carga, tenemos que crear otro folder en el directorio del proyecto que se llama Java, donde se encuentra el Engine.java. Luego, en ese folder podemos crear nuestra simulación. Es importante añadir las siguientes importaciones:

```
import io.gatling.javaapi.core.ChainBuilder;
import io.gatling.javaapi.core.FeederBuilder;
import io.gatling.javaapi.core.ScenarioBuilder;
import io.gatling.javaapi.core.Simulation;
import io.gatling.javaapi.http.HttpProtocolBuilder;

import static io.gatling.javaapi.core.CoreDsl.*;
import static io.gatling.javaapi.http.HttpDsl.http;
```

La clase de nuestro test tiene que heredar de Simulation. Tenemos que especificar el endpoint al que vamos a probar.

```
HttpProtocolBuilder httpProtocol =
    http.baseUrl("http://127.0.0.1:53347")
        .acceptHeader("application/json")
        .contentTypeHeader("application/json");
```

En este caso el URL debe ser el de la aplicación de Flask Para cargar un registro aleatorio de nuestro dataset, usamos un Feeder.

```
private static FeederBuilder.FileBased<Object> jsonFeeder =
    jsonFile("data/pokedex.json").random();
```

Para cada función, hicimos un chaibuilder para encadenar los procesos. Para enviar la información de un registro, utilizamos parámetros en el form del request y también el endpoint de la aplicación de Flask puede recibir un parámetro en la ruta, que en este caso puede ser el Id. Un ejemplo es el siguiente:

```
private static ChainBuilder getPokemonId =
    feed(jsonFeeder)
        .exec(http("Get one Pokemon #{Id}")
            .get("/getPokemon/#{Id}")
            .header("Content-Type", "application/x-www-form-urlencoded")
            .formParam("Id", "#{Id}")
            .formParam("Name", "#{Name}"));
```

Para correr la prueba, se necesita un escenario. Este debe ser una simulación representativa de cómo un usuario navegaría en el sistema. Para crear un escenario, se usa un ScenarioBuilder:

```

ScenarioBuilder scn = scenario("Database stress test with inserts").forever().on(
    pace(2)
        .feed(jsonFeeder)
        .exec(deletePokemon)
        .pause(2)
    );

```

Este escenario tiene el ciclo de forever porque de esta forma se pueda limitar su duración. Para terminar la configuración, se utiliza un setUp. Aquí se ponen ciertas características, como la cantidad de usuarios, los protocolos, y la duración.

```

{
    setUp(
        scn.injectOpen(
            nothingFor(5),
            rampUsers(20).during(30)
        )
    ).protocols(httpProtocol).maxDuration(900);
}

```

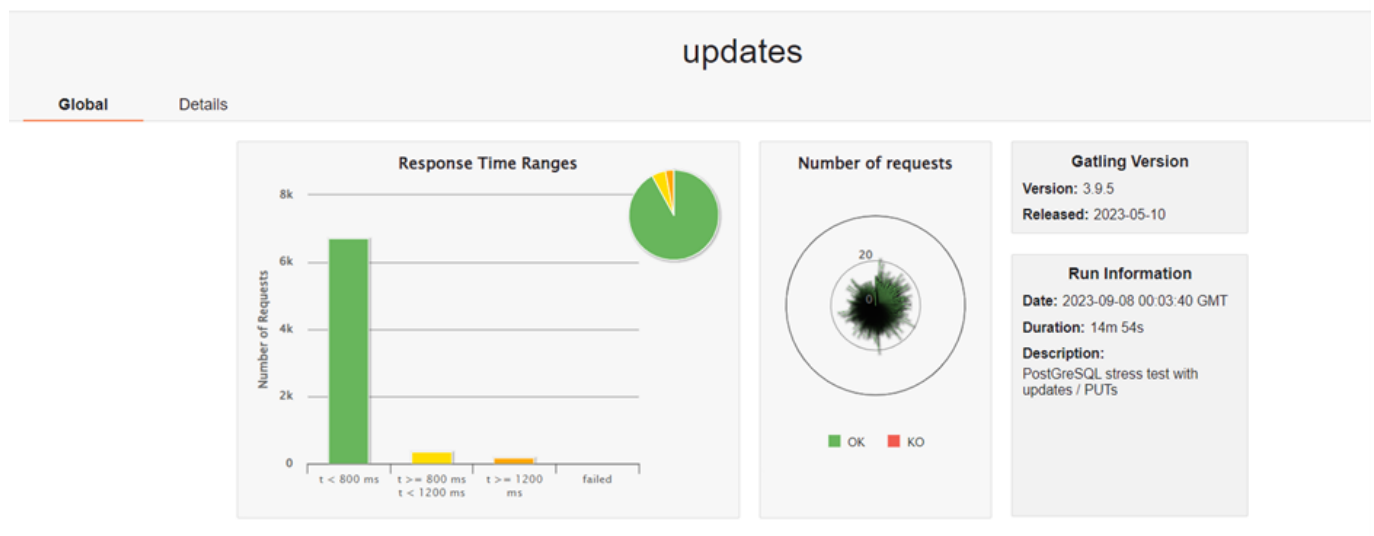
Ya con esto debería estar lista la prueba de carga. Para correrla, hay que ejecutar el archivo de Engine. Este pide cuál simulación correr y la ejecuta.

```

Choose a simulation number:
[0] Test.deletes
[1] Test.gets
[2] Test.inserts
[3] Test.testFlask
[4] Test.updates

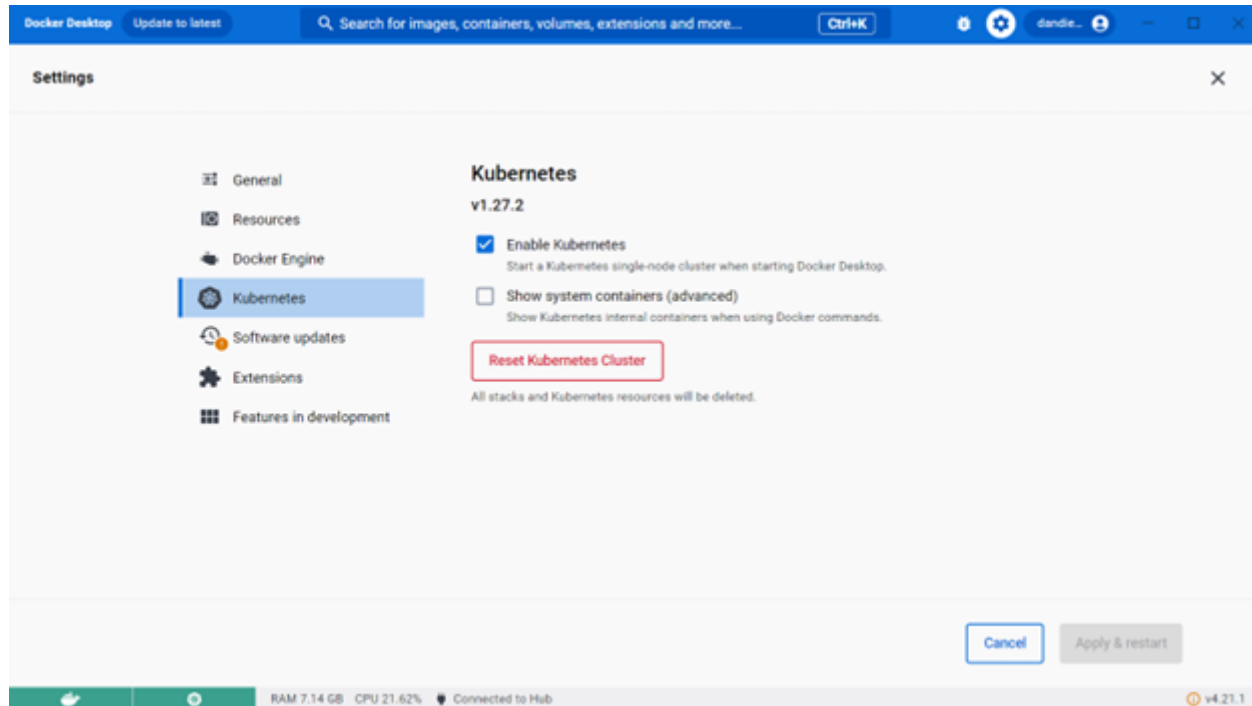
```

Gatling automáticamente genera un reporte, el cual se puede acceder mediante un link que devuelve al final o en los archivos del proyecto. Un ejemplo es el siguiente:



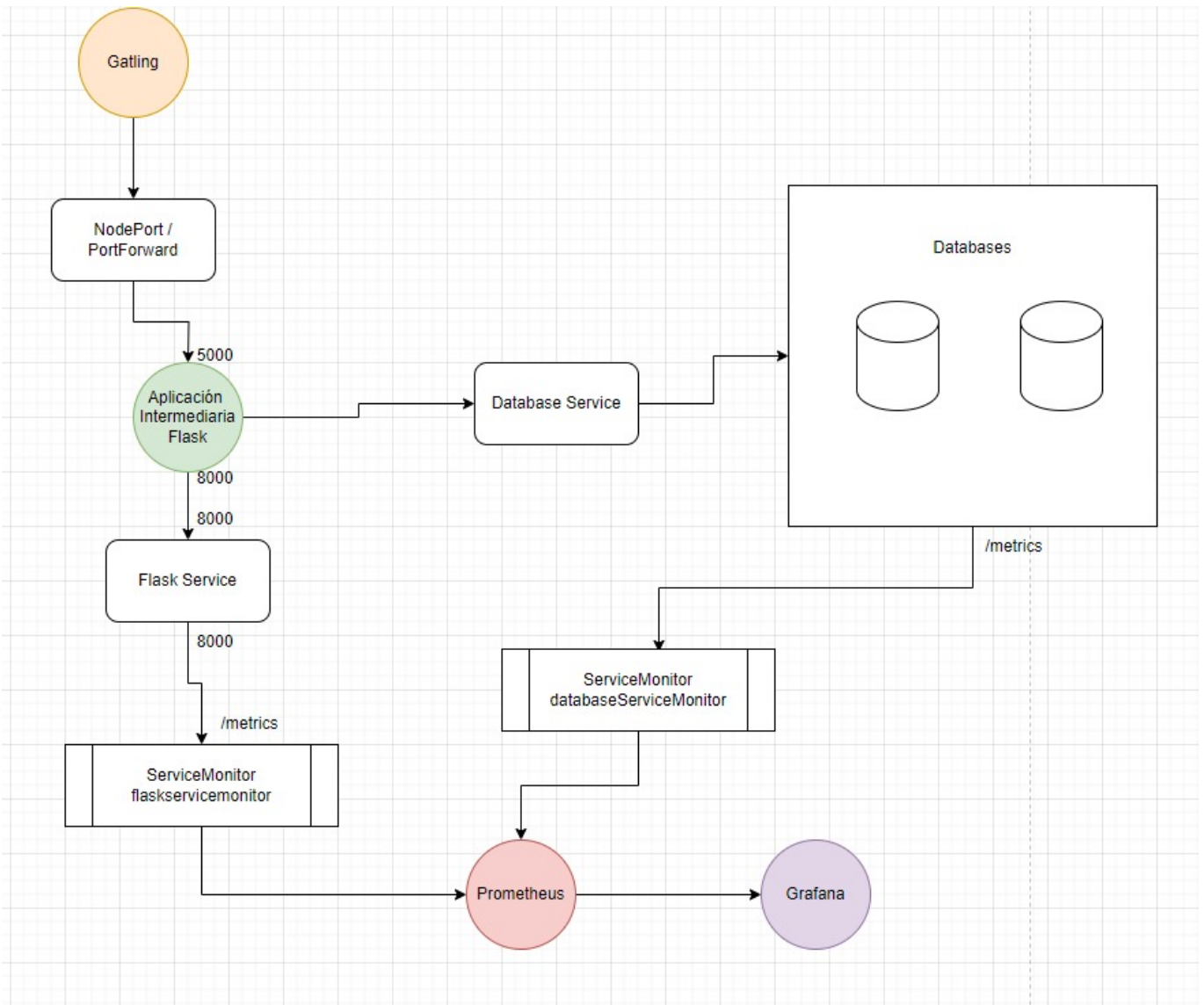
Para ejecutar la aplicación, es necesario instalar los helm charts de los componentes principales. Para esto, debemos tener las aplicaciones de Kubernetes y Helm instaladas. Instalar Kubernetes es muy sencillo si lo instalamos por medio de Docker Desktop.

Para instalarlo, debemos acceder a los settings haciendo click en la esquina superior derecha y activando el Kubernetes. Esto instala Kubernetes, y ya luego podemos acceder a él por medio de la herramienta en la línea de comandos, kubectl.



Luego, para instalar Helm, se puede acceder a la siguiente dirección: <https://helm.sh/>. Aquí, se podrán encontrar instrucciones para la instalación. Es especialmente fácil instalar Helm por medio de la herramienta chocolatey.

Instalación y uso



Para la instalación en general de cualquiera de las bases de datos con su aplicación intermediaria y con sus gráficos en Grafana se debe abrir una terminal en la carpeta de charts y correr los siguientes comandos en este orden y esperando entre cada instalación para dar tiempo a que inicien los pods:

```
helm install bootstrap bootstrap helm install monitoring-stack monitoring-stack helm
install databases databases helm install app app helm install grafana-config grafana-
config
```

Cabe destacar que cuando se vaya a utilizar Elasticsearch, se debe modificar el values.yaml en la carpeta de bootstrap. El campo de elasticsearch.enabled debe estar en true, ya que este instala el Elastic Cloud Operator. Para todas las demás bases de datos, este valor debe estar en **false**.

También, usualmente vamos a dejar activado el dashboard de grafana llamado flaskapp para poder monitorear las métricas de la aplicación intermediaria. Las otras dashboards correspondientes a las bases de datos deben estar en true si se desea visualizar sus métricas.

####Maria DB Para la instalación de Maria DB es necesario configurar los siguientes archivos dentro de la carpeta de charts: En la carpeta app, en el archivo de values, cambiar los valores de enabled de todas las bases de datos menos la de Maria DB a false, la de Maria DB debe de aparecer en true. En la carpeta de databases aplica lo mismo, buscamos el archivo de values y cambiamos los valores de enabled de las bases de datos que no deseamos comprobar, y dejamos solo en true la de Maria DB. De igual forma en la carpeta de

grafana-config en el archivo de valores cambiar los valores de enable para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

Maria DB Galera

Para la instalación de Maria DB Galera es necesario configurar los siguientes archivos dentro de la carpeta de charts: En la carpeta app, en el archivo de valores, cambiar los valores de enabled de todas las bases de datos menos la de Maria DB Galera a false, la de Maria DB Galera debe de aparecer en true. En la carpeta de databases aplica lo mismo, buscamos el archivo de valores y cambiamos los valores de enabled de las bases de datos que no deseamos comprobar, y dejamos solo en true la de Maria DB Galera. De igual forma en la carpeta de grafana-config en el archivo de valores cambiar los valores de enable para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

MongoDB

Para la instalación de MongoDB es necesario configurar los siguientes archivos dentro de la carpeta de charts: En la carpeta app, en el archivo de valores, cambiar los valores de enabled de todas las bases de datos menos la de Maria DB a false, la de MongoDB debe de aparecer en true. En la carpeta de databases aplica lo mismo, buscamos el archivo de valores y cambiamos los valores de enabled de las bases de datos que no deseamos comprobar, y dejamos solo en true la de MongoDB. De igual forma en la carpeta de grafana-config en el archivo de valores cambiar los valores de enable para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

PostgreSQL

Para la instalación de PostgreSQL, es necesario configurar los siguientes archivos dentro de la carpeta de charts. En la carpeta app, en el archivo de valores, cambiar los valores de enabled de todas las bases de datos menos la de PostgreSQL a false, la de PostgreSQL debe de aparecer en true. En la carpeta de databases aplica lo mismo, buscamos el archivo de valores y cambiamos los valores de enabled de las bases de datos que no deseamos comprobar, y dejamos solo en true la de PostgreSQL. De igual forma en la carpeta de grafana-config en el archivo de valores cambiar los valores de enable para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

PostgreSQL HA

Para la instalación de PostgreSQL HA es necesario configurar los siguientes archivos dentro de la carpeta de charts: En la carpeta app, en el archivo de valores, cambiar los valores de enabled de todas las bases de datos menos la de PostgreSQL HA a false, la de PostgreSQL HA debe de aparecer en true. En la carpeta de databases aplica lo mismo, buscamos el archivo de valores y cambiamos los valores de enabled de las bases de datos que no deseamos comprobar, y dejamos solo en true la de PostgreSQL HA. De igual forma en la carpeta de grafana-config en el archivo de valores cambiar los valores de enable para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

Elasticsearch

Para la instalación de Elasticsearch, es necesario configurar los siguientes archivos dentro de la carpeta de charts. En la carpeta de bootstrap, en el archivo de valores, cambiar el valor de `elasticsearch.enabled` a `true`. Esto instala el elastic operator. En la carpeta app, en el archivo de valores, cambiar los valores de `enabled` de todas las bases de datos menos la de Elasticsearch a `false`. La de Elasticsearch debe de aparecer en `true`. En la carpeta de databases aplica lo mismo, buscamos el archivo de valores y cambiamos los valores de `enabled` de las bases de datos que no deseamos comprobar, y dejamos solo en `true` la de Elasticsearch. Aquí podríamos activar Kibana si quisiéramos utilizar la interfaz gráfica. Esto lo hacemos modificando el valor `enabled` debajo de kibana a `true`. De igual forma en la carpeta de grafana-config en el archivo de valores cambiar los valores de `enable` para que solo las gráficas deseadas sean visualizadas. Una vez se realizan estos procesos se puede continuar con la instalación mencionada al inicio de esta sección.

Configuración de las herramientas

Para la configuración de las aplicaciones intermediarias se utilizó una imagen de Docker en la cual se instalaban las librerías para conectarse con cada base de datos. Lo mismo se utilizó para instalar el Prometheus Client y Flask. La librería para conectarse con MongoDB que se usó es `Flask_Pymongo`, para conectarse con MariaDB y MariaDB Galera se llama `MariaDB`, para conectarse con PostgreSQL y PostgreSQL-HA es `pg8000` y para conectarse con Elasticsearch se utilizó `elasticsearch`.

Para conectar todas las aplicaciones de Flask con Prometheus, se utilizó la librería de [Prometheus Python Client](#). Esta nos permite abrir un servicio de http que expone métricas a Prometheus en la ruta `/metrics` en el puerto que le definamos en la función:

```
start_http_server(8000)
```

Aquí definimos el servidor en el puerto 8000, por lo que también lo exponemos en la configuración del YAML.

Luego, debemos definir un `ServiceMonitor` de Prometheus que se va a encargar de hacer el scrape a la aplicación. Sin embargo, estos trabajan con servicios, por lo que debemos definir otro servicio de tipo `ClusterIP` para comunicar estos dos componentes. Al servicio `ClusterIP`, lo relacionamos con la aplicación de flask por medio del label `"app: nombre"`. Le definimos que el puerto 8000 va a ser el puerto de métricas. Le agregamos las siguientes anotaciones para señalarle a Prometheus que esto es un endpoint que tiene que hacerle scraping:

```
prometheus.io/port: "metrics"
prometheus.io/scrape: "true"
```

El label de `app.kubernetes.io/part-of: dms` se usa también para relacionar el servicio y el `serviceMonitor`. Finalmente, utilizamos el CRD definido por el `prometheus-operated` para crear el `ServiceMonitor` en el namespace de `monitoring`. Adicionalmente, la configuración determina que se revisará ese endpoint cada 10 segundos en el puerto definido en el servicio de `"metrics"`. Tutorial utilizado para establecer el `ServiceMonitor`: [Fuente](#).

Ya configurado esto, al inicio de cada aplicación de Flask, se definió la métrica de tipo Counter llamada `flask_http_requests`, la cual lleva la cuenta de la cantidad total de requests http que recibe el API. Prometheus

le agrega el sufijo de `_total` al ser un contador, por lo que en el cliente de prometheus aparece como

```

58  apiVersion: v1
59  kind: Service
60  metadata:
61    name: flaskservice
62    namespace: default
63    annotations:
64      prometheus.io/port: "metrics"
65      prometheus.io/scrape: "true"
66    labels:
67      app.kubernetes.io/part-of: dms
68  spec:
69    type: ClusterIP
70    selector:
71      app: {{ .Values.config.elasticsearch.name
72    ports:
73      - port: 8000
74        targetPort: 8000
75        protocol: TCP
76        name: "metrics"
77  ---
78  apiVersion: monitoring.coreos.com/v1
79  kind: ServiceMonitor
80  metadata:
81    name: flaskservicemonitor
82    namespace: monitoring
83  spec:
84    endpoints:
85      - interval: 10s
86        port: metrics
87    namespaceSelector:
88      matchNames:
89        - default
90    selector:
91      matchLabels:
92        app.kubernetes.io/part-of: dms
93  {{- end }}

```

`flask_http_requests_total`.

Mongo DB

Helm Charts

Para conectarse con MongoDB, se establecieron algunas variables en la configuración del Helm Chart de la aplicación. El pod se va a llamar `mongodb`, el namespace va a ser `mongodb`, la base de datos se va a llamar `Pokemon`, el usuario es `admin`, la contraseña es `admin` y el servicio al que se conecta es `databases-mongodb-headless`. De esta forma, la aplicación va a poder reconocer las bases de datos de Mongo. Las variables se establecieron para que la aplicación agarre los valores para conectarse de ahí.

En la configuración de las bases de datos de MongoDB, se utilizó una configuración similar. Se creó el usuario de `admin`, la contraseña de `admin` y la base de datos de `Pokemon`. Esto se realizó en el `values.yaml` del Helm Chart de `Databases`. El namespace se establece como `mongodb` para que las bases de datos se puedan

comunicar con la aplicación de Flask. el replicaCount se usa para que Kubernetes cree 3 réplicas de la base de datos. Tienen un límite de CPU de 0.6 y 2 gigas de Memoria. El serviceMonitor se estableció en el namespace de monitoring para que se comuniquen con Prometheus.

```
mongodb:
  enabled: false
  auth:
    usernames: ["admin"]
    passwords: ["admin"]
    databases: ["Pokemon"]
  global:
    namespaceOverride: mongodb
  architecture: replicaset
  replicaCount: 3
  resources:
    limits:
      cpu: "0.6"
      memory: "2Gi"
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      namespace: monitoring
```

Para la configuración de los dashboards de Grafana, se accedió a Prometheus para verificar las métricas de MongoDB. Luego, con los nombres correctos, se actualizó el siguiente dashboard para que se desplegaran valores: <https://grafana.com/grafana/dashboards/2583-mongodb/> El dashboard actualizado se encuentra en los dashboards del grafana-config

Aplicación intermediaria

En el programa de Flask en sí, para conectarse se utilizan las siguientes instrucciones:

```
app.config["MONGO_URI"] = 'mongodb://' + environ['MONGODB_USERNAME'] + ':' +
environ['MONGODB_PASSWORD'] + '@' + environ['MONGODB_HOSTNAME'] + ':27017/' +
environ['MONGODB_DATABASE']

mongo = PyMongo(app)
db = mongo.db
```

Estas crean la instancia de la base de datos en la aplicación. La base de datos de Pokémon se crea si no existe.

Maria DB

Helm Charts

Los Helm charts de Maria DB se encuentran en la carpeta de charts bajo el nombre de databases. Primero se mostrarán los Helm Charts de la configuración de la Base de Datos:

```
mariadb:
  enabled: false
  architecture: replication
  primary:
    resources:
      limits:
        cpu: "1"
        memory: "4Gi"
  secondary:
    replicaCount: 2
    resources:
      limits:
        cpu: "0.5"
        memory: "2Gi"
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      namespace: monitoring
```

En este caso se le indica a la base que está trabajando con un modo de replicación, y se le indica los recursos que debe darle tanto a las réplicas como al primario y cuantas instancias queremos de replicación. Y finalmente se le habilitan las métricas.

Aplicación intermediaria

Seguidamente se mostrará los Helm Charts de la aplicación intermedia:

```
config:
  flaskmariadb:
    enabled: false
    replicas: 1
    name: flaskmariadb
    image: dado715/flask-mariadb:latest
```

```

{{- if .Values.config.flaskmariadb.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.config.flaskmariadb.name }}
  labels:
    app: {{ .Values.config.flaskmariadb.name }}
spec:
  replicas: {{ .Values.config.flaskmariadb.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.config.flaskmariadb.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.config.flaskmariadb.name }}
    spec:
      containers:
        - name: {{ .Values.config.flaskmariadb.name }}
          image: {{ .Values.config.flaskmariadb.image }}
          env:
            - name: MDB_USERNAME
              value: "root"
            - name: MDB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: databases-mariadb
                  key: mariadb-root-password
            - name: MDB_HOST
              value: databases-mariadb-primary
            - name: MDB_DATABASE
              value: "pokemon"

```

```

apiVersion: v1
kind: Service
metadata:
  name: flask-port-mariadb
  namespace: default
  labels:
    app: {{ .Values.config.flaskmariadb.name }}
spec:
  selector:
    app: {{ .Values.config.flaskmariadb.name }}
  ports:
    - name: http
      protocol: TCP
      port: 5000
      targetPort: 5000
      nodePort: 30091
  type: NodePort

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: flaskservice
  namespace: default
  annotations:
    prometheus.io/port: "metrics"
    prometheus.io/scrape: "true"
  labels:
    app.kubernetes.io/part-of: dms
spec:
  type: ClusterIP
  selector:
    app: {{ .Values.config.flaskmariadb.name }}
  ports:
    - port: 8000
      targetPort: 8000
      protocol: TCP
      name: "metrics"
---

```

```

---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: flaskservicemonitor
  namespace: monitoring
spec:
  endpoints:
    - interval: 10s
      port: metrics
  namespaceSelector:
    matchNames:
      - default
  selector:
    matchLabels:
      app.kubernetes.io/part-of: dms
{{- end }}

```

En esta primera imagen se puede observar los valores que va a tomar el template del deployment para Kubernetes. En este caso se definen valores para crear el deployment con los nombres que se definen en los valores y seguidamente se crean los servicios para exportar las métricas a Prometheus y el NodePort para exponerlo a la máquina host.

PostgreSQL

Helm Charts

La configuración del Helm Chart está en el archivo de valores en la carpeta de /charts/databases. Esto se hizo con base en el repositorio de [Bitnami](#). En el repositorio, podemos encontrar todos los valores de configuración posible. La mayoría se podrían dejar con sus valores determinados. Sin embargo, si es necesario especificar algunos valores, especialmente para configurar el ServiceMonitor y exponer las métricas.


```
postgresql:
  enabled: false
  primary:
    resources:
      limits:
        cpu: "1"
        memory: "4Gi"
  auth:
    database: "Pokemon"
    username: "admin"
    password: "admin"
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      namespace: monitoring
```

En esta configuración, estamos estableciendo que la instancia de PostGreSQL va a tener acceso a 1 CPU y a 4GB de memoria. Adicionalmente, es necesario especificar la base de datos a la cual se conectará y definir un usuario. En este caso estamos definiendo al usuario "admin" y su contraseña. Finalmente, es necesario habilitar las métricas de Prometheus, y para esto debemos habilitar el ServiceMonitor, el cual estará en el namespace de monitoring, donde se encuentra la instancia de Prometheus.

Aplicación intermediaria

La configuración de la aplicación intermediaria se encuentra esencialmente en el archivo postgresql.yaml en la carpeta /charts/app/templates. Esta configuración es parametrizada con la ayuda del archivo values.yaml en la carpeta /charts/app. En el template, definimos el deployment que va a controlar al contenedor de la aplicación. En este le definimos las variables de entorno y exponemos los puertos que se usarán: el 5000 para Flask y el API y el 8000 para las métricas de Prometheus. Adicionalmente, se definen los servicios para el ServiceMonitor ya descritos y un servicio de tipo NodePort para exponer la aplicación al localhost.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.config.postgresql.name }}
  labels:
    app: {{ .Values.config.postgresql.name }}
spec:
  replicas: {{ .Values.config.postgresql.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.config.postgresql.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.config.postgresql.name }}
    spec:
      containers:
```

```
- name: {{ .Values.config.postgresql.name }}
  image: {{ .Values.config.postgresql.image }}
  ports:
    - name: "flask"
      containerPort: 5000
    - name: "metrics"
      containerPort: 8000
  env:
    - name: PGDATABASE
      value: {{ .Values.config.postgresql.database }}
    - name: PGUSER
      value: {{ .Values.config.postgresql.user }}
    - name: PGPASSWORD
      value: {{ .Values.config.postgresql.password }}
    - name: PGSERVICE
      value: {{ .Values.config.postgresql.service }}
```

Este deployment solo corre cuando el valor de postgresql.enabled en el values.yaml es igual a true. En el values.yaml, le mandamos cuál es la base de datos que usará para trabajar, el usuario que usará para conectarse y su contraseña y finalmente el servicio que expone la instancia de PostgreSQL para conectarse a la base de datos.

```
postgresql:
  enabled: false
  replicas: 1
  name: postgreflask
  image: dandiego235/postgreflask
  database: "Pokemon"
  user: "admin"
  password: "admin"
  service: "databases-postgresql"
```

Maria DB Galera

Helm Charts

La configuracion de Helm Chart se encuentra en la carpeta /charts/databases, en el archivo values.yaml, este fue creado utilizando de base la documentacion de [Artifact HUB](#).

```
mariadb-galera:
  enabled: false
  replicaCount: 2
  metrics:
    enabled: true
  serviceMonitor:
    enabled: true
    namespace: monitoring
```

En su configuración se declara que va a tener 2 réplicas y adicionalmente estamos habilitando las métricas de prometheus habilitando el serviceMonitor y posteriormente se le asigna el namespace de monitoring en el cual se encuentra la instancia de Prometheus.

Aplicación intermediaria

La configuración de la aplicación intermediaria, ubicada en /charts/app/templates/flask-mariadbgal.yaml, define un Deployment para controlar el ciclo de vida del contenedor de la aplicación. El nombre, las etiquetas y otros parámetros clave provienen del archivo values.yaml en la carpeta /charts/app. En este Deployment, la aplicación utiliza el puerto 5000 para Flask y API y el puerto 8000 para las métricas de Prometheus. Además, se establecen variables de entorno para conectar a la base de datos MariaDB Galera.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.config.flaskmariadbgal.name }}
  labels:
    app: {{ .Values.config.flaskmariadbgal.name }}
spec:
  replicas: {{ .Values.config.flaskmariadbgal.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.config.flaskmariadbgal.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.config.flaskmariadbgal.name }}
    spec:
      containers:
        - name: {{ .Values.config.flaskmariadbgal.name }}
          image: {{ .Values.config.flaskmariadbgal.image }}
          ports:
            - name: "flask"
              containerPort: 5000
            - name: "metrics"
              containerPort: 8000
          env:
            - name: MDB_USERNAME
              value: "root"
            - name: MDB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: databases-mariadb-galera
                  key: mariadb-root-password
            - name: MDB_HOST
              value: databases-mariadb-galera-headless
            - name: MDB_DATABASE
              value: "pokemon"
```

Dentro del archivo values.yaml se le define del Deployment el cual es "flaskmariadbgal", y se le indica el contenedor que utilizara dentro del pod el cual es dado715/flask-mariadb:latest del repositorio de DockerHub.

```
flaskmariadbgal:
  enabled: false
  replicas: 1
  name: flaskmariadbgal
  image: dado715/flask-mariadb:latest
```

PostgreSQL HA

Helm Charts

Los Helm charts de PostgreSQL HA se encuentran en la carpeta de charts bajo el nombre de databases. Primero se mostrarán los Helm Charts de la configuración de la Base de Datos:

```
postgresql-ha:
  enabled: false
  postgresql:
    database: "pokemon"
  pgpool:
    pgpool:
      maxPool: 25
  persistence:
    enabled: false
    size: "2Gi"
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      namespace: monitoring
```

Los parámetros elegidos para el correcto funcionamiento de la base de datos fueron, crearle una base de datos, en el gpool que es un pool de conexiones ponerle una cantidad de 25 para poder mantener los 20 usuarios que envían requests al sistema y la configuración de las métricas.

Aplicación intermediaria

Seguidamente se mostrará los Helm Charts de la aplicación intermedia:

```
postgresqlha:
  enabled: false
  name: flaskpostgresqlha
  image: dado715/flask-postgresqlha:latest
```

```

{{- if .Values.config.postgresqlha.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.config.postgresqlha.name }}
  labels:
    app: {{ .Values.config.postgresqlha.name }}
spec:
  replicas: {{ .Values.config.postgresqlha.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.config.postgresqlha.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.config.postgresqlha.name }}
    spec:
      containers:
        - name: {{ .Values.config.postgresqlha.name }}
          image: {{ .Values.config.postgresqlha.image }}
          ports:
            - containerPort: 5000
          env:
            - name: PG_USER
              value: "postgres"
            - name: PG_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: databases-postgresql-ha-postgresql
                  key: password
            - name: PG_HOST
              value: databases-postgresql-ha-pgpool
            - name: PG_DATABASE
              value: "pokemon"

```

```

---
- name: PG_DATABASE
  value: "pokemon"

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: flaskportpgha
  namespace: default
  labels:
    app: {{ .Values.config.postgresqlha.name }}
spec:
  type: NodePort
  selector:
    app: {{ .Values.config.postgresqlha.name }}
  ports:
    - name: http
      protocol: "TCP"
      port: 5000
      targetPort: 5000
      nodePort: 30094

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: flaskservice
  namespace: default
  annotations:
    prometheus.io/port: "metrics"
    prometheus.io/scrape: "true"
  labels:
    app.kubernetes.io/part-of: dms
spec:

```

```
spec:
  type: ClusterIP
  selector:
    app: {{ .Values.config.postgresqlha.name }}
  ports:
    - port: 8000
      targetPort: 8000
      protocol: TCP
      name: "metrics"
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: flaskservicemonitor
  namespace: monitoring
spec:
  endpoints:
    - interval: 10s
      port: metrics
  namespaceSelector:
    matchNames:
      - default
  selector:
    matchLabels:
      app.kubernetes.io/part-of: dms
{{- end }}
```

En esta

primera imagen se puede observar los valores que va a tomar el template del deployment para Kubernetes. En este caso se definen valores para crear el deployment con los nombres que se definen en los values y seguidamente se crean los servicios para exportar las métricas a Prometheus y el NodePort para exponerlo a la máquina host.

Elasticsearch

Helm Charts

El primer Helm chart usado es el del Elastic Operator, el cual podemos encontrar en el [repositorio de Helm de Elastic](#). Este se instala con el Helm chart de Bootstrap. En este Helm chart no se utiliza directamente una dependencia propia de Elasticsearch, ya que se utiliza el mismo operador. Aquí se establecen dos templates, uno para Elasticsearch y otro para Kibana.

El segundo Helm chart usado en la instalación es el de Databases. En el template de elastic.yaml, tenemos lo siguiente:

```
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: {{ .Values.elasticsearch.elastic.name }}
spec:
  version: {{ .Values.elasticsearch.elastic.version }}
  http:
    tls:
      selfSignedCertificate:
        disabled: true
  nodeSets:
```

```
- name: master
  count: {{ .Values.elasticsearch.elastic.mastercount }}
  podTemplate:
    spec:
      containers:
        - name: elasticsearch
          resources:
            requests:
              memory: 2Gi
              cpu: 1
            limits:
              memory: 2Gi
      config:
        node.roles: [master]
        node.store.allow_mmap: false
- name: data
  count: {{ .Values.elasticsearch.elastic.datacount }}
  podTemplate:
    spec:
      containers:
        - name: elasticsearch
          resources:
            requests:
              memory: 2Gi
              cpu: 1
            limits:
              memory: 2Gi
      config:
        node.roles: [data]
        node.store.allow_mmap: false
```

Aquí se especifica que se tendrá un número de nodos maestros con 1 CPU y 2GB de RAM y otra cantidad de nodos de datos con 1 CPU y 2GB de RAM. Estos valores se establecen en el archivo de values.yaml del helm chart:

```
elasticsearch:
  enabled: true
  elastic:
    version: 8.6.1
    mastercount: 1
    datacount: 2
    name: ic4302
  kibana:
    enabled: false
    version: 8.6.1
    replicas: 1
    name: ic4302
```

Aquí se establece la versión de elasticsearch y que vamos a tener 1 nodo maestro y 2 nodos de datos. No se logró tener 3 nodos de datos por recursos limitados del computador en la prueba.

Pods										
8 items										
Namespace: default										
Search Pods...										
☐ Name	Namespace	Cont...	CPU	Memory	Restarts	Controlled By	Node	QoS	Age	Status
☐ monitoring-stack-node-exporter-k9ngl	default	●	N/A	N/A	0	DaemonSet	docker-de	BestEffort	63m	Running
☐ monitoring-stack-kube-state-metrics-84f9d4dfc-kcmnd	default	●	N/A	N/A	0	ReplicaSet	docker-de	BestEffort	63m	Running
☐ ic4302-es-master-0	Warning	●		1A	0	StatefulSet	docker-de	Burstable	68s	Running
☐ ic4302-es-data-2	Warning: 0/1 nodes are available: 1 Insufficient memory, preemption: 0/1 nodes are available: 1 No preemption victims found for incoming pod.									
☐ ic4302-es-data-1	default	●	N/A	N/A	0	StatefulSet	docker-de	Burstable	68s	Running
☐ ic4302-es-data-0	default	●	N/A	N/A	0	StatefulSet	docker-de	Burstable	68s	Running
☐ elastic-operator-0	default	●	N/A	N/A	2	StatefulSet	docker-de	Burstable	63m	Running
☐ databases-elasticprometheus-564864fcf-rvzxx	default	●	N/A	N/A	0	ReplicaSet	docker-de	BestEffort	70s	Running

☐ Name	Namespace	Cont...	CPU	Memory	Restarts	Controlled By	Node	QoS	Age	Status
☐ monitoring-stack-node-exporter-k9ngl	default	●	N/A	N/A	0	DaemonSet	docker-de	BestEffort	71m	Running
☐ monitoring-stack-kube-state-metrics-84f9d4dfc-kcmnd	default	●	N/A	N/A	0	ReplicaSet	docker-de	BestEffort	71m	Running
☐ ic4302-es-master-0	default	●	N/A	N/A	0	StatefulSet	docker-de	Burstable	2m33s	Running
☐ ic4302-es-data-1	default	●	N/A	N/A	0	StatefulSet	docker-de	Burstable	2m33s	Running
☐ ic4302-es-data-0	default	●	N/A	N/A	0	StatefulSet	docker-de	Burstable	2m33s	Running
☐ elasticflask-5dc5cb977f-kcgng	default	●	N/A	N/A	0	ReplicaSet	docker-de	BestEffort	7s	Running
☐ elastic-operator-0	default	●	N/A	N/A	2	StatefulSet	docker-de	Burstable	71m	Running
☐ databases-elasticprometheus-564864fcf-rvzxx	default	●	N/A	N/A	0	ReplicaSet	docker-de	BestEffort	30s	Running

Otro helm chart necesario es el de prometheus-elasticsearch-exporter, el cual se utiliza para recolectar métricas y exponerlas a Prometheus. Esta dependencia se descarga en el archivo Charts.yaml:

```
- name: prometheus-elasticsearch-exporter
  alias: elasticprometheus
  version: "5.2.0"
  repository: https://prometheus-community.github.io/helm-charts
  condition: elasticsearch.enabled
```

A esta dependencia se le pone el alias de elasticprometheus y se activa cuando elasticsearch está activado. Esta dependencia tiene la siguiente configuración:

```
elasticprometheus:
  env:
    ES_USERNAME: elastic
  extraEnvSecrets:
    ES_PASSWORD:
      secret: ic4302-es-elastic-user
      key: elastic
  es:
    uri: http://ic4302-es-http:9200
  serviceMonitor:
    enabled: true
    namespace: monitoring
```

En esta configuración se le pasan las variables de entorno necesarias para la autenticación con elasticsearch. La contraseña se le pasa por medio el secret generado por el operator. Adicionalmente, se le envía la dirección del servicio que expone elasticsearch para conectarse. Finalmente, se establece la creación de un ServiceMonitor en el namespace de Monitoring para que Prometheus pueda monitorear las métricas generadas.

Aplicación intermediaria

En el template del deployment de la aplicación intermediaria, se establecen las variables de entorno requeridas para conectarse con elasticsearch:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.config.elasticsearch.name }}
  labels:
    app: {{ .Values.config.elasticsearch.name }}
spec:
  replicas: {{ .Values.config.elasticsearch.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.config.elasticsearch.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.config.elasticsearch.name }}
    spec:
      containers:
        - name: {{ .Values.config.elasticsearch.name }}
          image: {{ .Values.config.elasticsearch.image }}
          ports:
            - name: "flask"
              containerPort: 5000
            - name: "metrics"
              containerPort: 8000
      env:
        - name: ESINDEX
          value: {{ .Values.config.elasticsearch.index }}
        - name: ESUSERNAME
          value: {{ .Values.config.elasticsearch.username }}
        - name: ESPASSWORD
          valueFrom:
            secretKeyRef:
              name: ic4302-es-elastic-user
              key: elastic
              optional: false
        - name: ESENDPOINT
          value: {{ .Values.config.elasticsearch.service }}
```

En la configuración de estos valores en el values.yaml, se establecen los siguientes valores:

```
elasticsearch:
  enabled: true
  name: elasticflask
  image: dandiego235/elasticflask
  index: "pokemones"
  username: "elastic"
  service: "ic4302-es-http"
```

Aquí se establece la imagen del contenedor, el índice que se crea en elasticsearch, el nombre de usuario y finalmente el servicio donde se puede conectar a los pods de elasticsearch.

Pruebas de carga realizadas

El dataset utilizado para las pruebas es uno que contiene a todos los Pokémones. Cada registro tiene el siguiente formato:

```
[
  {
    "Id": "#{Id}",
    "Name": "#{Name}",
    "Type1": "#{Type1}",
    "Type2": "#{Type2}",
    "Category": "#{Category}",
    "Heightf": "#{Heightf}",
    "Heightm": "#{Heightm}",
    "Weightlbs": "#{Weightlbs}",
    "Weightkg": "#{Weightkg}",
    "CaptureRate": "#{CaptureRate}",
    "EggSteps": "#{EggSteps}",
    "ExpGroup": "#{ExpGroup}",
    "Total": "#{Total}",
    "HP": "#{HP}",
    "Attack": "#{Attack}",
    "Defense": "#{Defense}",
    "SpAttack": "#{SpAttack}",
    "SpDefense": "#{SpDefense}",
    "Speed": "#{Speed}"
  }
]
```

Unos ejemplos son los siguientes:

```
{
  "Id": "001",
  "Name": "Bulbasaur",
  "Type1": "Grass",
  "Type2": "Poison",
  "Category": "Seed Pokémon",
  "Heightf": "2'04\"",
  "Heightm": "0.7",
  "Weightlbs": "15.2",
  "Weightkg": "6.9",
  "CaptureRate": "45",
  "EggSteps": "5120",
  "ExpGroup": "Medium Slow",
  "Total": "318",
  "HP": "45",
```

```
        "Attack": "49",
        "Defense": "49",
        "SpAttack": "65",
        "SpDefense": "65",
        "Speed": "45"
    },
    {
        "Id": "002",
        "Name": "Ivysaur",
        "Type1": "Grass",
        "Type2": "Poison",
        "Category": "Seed Pokémon",
        "Heightf": "3'03\"",
        "Heightm": "1",
        "Weightlbs": "28.7",
        "Weightkg": "13",
        "CaptureRate": "45",
        "EggSteps": "5120",
        "ExpGroup": "Medium Slow",
        "Total": "405",
        "HP": "60",
        "Attack": "62",
        "Defense": "63",
        "SpAttack": "80",
        "SpDefense": "80",
        "Speed": "60"
    },
    {
        "Id": "003",
        "Name": "Venusaur",
        "Type1": "Grass",
        "Type2": "Poison",
        "Category": "Seed Pokémon",
        "Heightf": "6'07\"",
        "Heightm": "2",
        "Weightlbs": "220.5",
        "Weightkg": "100",
        "CaptureRate": "45",
        "EggSteps": "5120",
        "ExpGroup": "Medium Slow",
        "Total": "525",
        "HP": "80",
        "Attack": "82",
        "Defense": "83",
        "SpAttack": "100",
        "SpDefense": "100",
        "Speed": "80"
    }
]
```

En total se realizaron 5 pruebas. Estas se encuentran en el proyecto de Gatling en el folder de Test: Inserts: Realiza inserts de Pokémons aleatorios por 15 minutos. El ChainBuilder utilizado es el siguiente:

```
private static ChainBuilder addPokemon =
    feed(jsonFeeder)

        .exec(http("Add new Pokemon - #{Name}")
            .post("/postPokemon")
            .header("Content-Type", "application/x-www-form-
urlencoded")

            .formParam("Id", "#{Id}")
            .formParam("Name", "#{Name}")
            .formParam("Type1", "#{Type1}")
            .formParam("Type2", "#{Type2}")
            .formParam("Category", "#{Category}")
            .formParam("Heightf", "#{Heightf}")
            .formParam("Heightm", "#{Heightm}")
            .formParam("Weightlbs", "#{Weightlbs}")
            .formParam("Weightkg", "#{Weightkg}")
            .formParam("CaptureRate", "#{CaptureRate}")
            .formParam("EggSteps", "#{EggSteps}")
            .formParam("ExpGroup", "#{ExpGroup}")
            .formParam("Total", "#{Total}")
            .formParam("HP", "#{HP}")
            .formParam("Attack", "#{Attack}")
            .formParam("Defense", "#{Defense}")
            .formParam("SpAttack", "#{SpAttack}")
            .formParam("SpDefense", "#{SpDefense}")
            .formParam("Speed", "#{Speed}")
        );
```

El header es un Content-Type para especificar el tipo de datos que envía el request. En este caso es un form con varios parámetros. Cada parámetro contiene información del Pokémon, como el número identificador, el nombre, los tipos y la categoría. El request es de tipo Post porque ese es el que recibe el endpoint de Flask.

- Gets: Realiza consultas de Pokémones aleatorios por 15 minutos. De esta prueba hay dos ChainBuilders:

```
private static ChainBuilder getPokemonId =
    feed(jsonFeeder)
        .exec(http("Get one Pokemon #{Id}")
            .get("/getPokemon/#{Id}")
            .header("Content-Type", "application/x-www-form-urlencoded")
            .formParam("Id", "#{Id}")
            .formParam("Name", "#{Name}"));

private static ChainBuilder getAllPokemon =
    exec(http("Get all Pokemon")
        .get("/getAllPokemon"));
```

El ChainBuilder de getPokemonId realiza un get de solo un Pokémon, el cual envía por medio de la ruta del endpoint de Flask y también por parámetros del request. El ChainBuilder de getAllPokémon envía un request

de get al endpoint de Flask. Este endpoint retorna todos los Pokémones almacenados en la base de datos y toda su información, similar a un SELECT * de una base de datos relacional.

- Updates: Actualiza Pokémones aleatorios por 15 minutos. El ChainBuilder es el siguiente:

```
private static ChainBuilder updatePokemon =
    feed(jsonFeeder)

        .exec(http("Update new Pokemon - #{Name}")
            .put("/putPokemon/#{Id}")
            .header("Content-Type", "application/x-www-form-
urlencoded")

            .formParam("Id", "#{Id}")
            .formParam("Name", "#{Name}")
            .formParam("Type1", "#{Type1}")
            .formParam("Type2", "#{Type2}")
            .formParam("Category", "#{Category}")
            .formParam("Heightf", "#{Heightf}")
            .formParam("Heightm", "#{Heightm}")
            .formParam("Weightlbs", "#{Weightlbs}")
            .formParam("Weightkg", "#{Weightkg}")
            .formParam("CaptureRate", "#{CaptureRate}")
            .formParam("EggSteps", "#{EggSteps}")
            .formParam("ExpGroup", "#{ExpGroup}")
            .formParam("Total", "#{Total}")
            .formParam("HP", "#{HP}")
            .formParam("Attack", "#{Attack}")
            .formParam("Defense", "#{Defense}")
            .formParam("SpAttack", "#{SpAttack}")
            .formParam("SpDefense", "#{SpDefense}")
            .formParam("Speed", "#{Speed}")

        );
```

Este ChainBuilder envía el Pokémon a modificar por medio de la ruta del endpoint de Flask. El request es un PUT. La información nueva la envía por parámetros. Es posible que para un Pokémon, el Feeder escoja la misma información de Pokémon, por lo que la actualización se haría por los mismos datos. Sin embargo, igualmente en el endpoint de Flask sí se realiza la actualización en la base de datos.

- Deletes: Elimina Pokémones aleatorios por 15 minutos. El ChainBuilder es el siguiente:

```
private static ChainBuilder deleteLastPostedPokemon =
    feed(jsonFeeder)

        .exec(http("Delete Pokemon - #{Name}")
            .delete("/deletePokemon/#{Id}")
            .header("Content-Type", "application/x-www-form-
urlencoded")

        );
```

Este ChainBuilder envía un request de DELETE al endpoint de Flask. El Pokémon a eliminar se manda por la ruta del endpoint. En nuestra implementación de las bases de datos, los Pokémones se almacenan con un Id generado por la base de datos. Por lo tanto, es posible que haya múltiples registros del mismo Pokémon. En la aplicación de Flask se procura que solo se borre un registro, al limitar la cantidad de borrado. De esta forma, si el Pokémon vuelve a aparecer en el escenario, se puede borrar nuevamente. Si se borraran todos los registros con el id que envía el request, solo se podría tener un máximo de alrededor de 700 eliminaciones, ya que esa es la cantidad de registros que hay en el dataset.

- testFlask: Realiza todas las operaciones en Pokémones aleatorios por 15 minutos. Utiliza todos los ChainBuilders mencionados anteriormente. El escenario que tiene es:

```
ScenarioBuilder scn = scenario("Database stress test with every request
type").forever().on(
    pace(15)
        .feed(jsonFeeder)
        .exec(addPokemon)
        .pause(2)
        .feed(jsonFeeder)
        .exec(getPokemonId)
        .pause(2)
        .feed(jsonFeeder)
        .exec(updatePokemon)
        .pause(2)
        .exec(getAllPokemon)
        .pause(2)
        .feed(jsonFeeder)
        .exec(deleteLastPostedPokemon)
        .pause(2)
);
```

En este escenario, se realizan las operaciones en el orden de insert, get, update y delete. De esta manera, un registro se puede consultar y actualizar antes de ser eliminado. Se utiliza un feeder para cambiar el registro escogido para cada operación. Se pone una pausa entre cada operación para evitar errores o saturación del servidor y también para simular de una manera más realista el tiempo entre requests de usuarios.

Resultado de las pruebas en cada base de datos

Maria DB

Para la base de datos de Maria DB, con las pruebas realizadas se obtuvieron las siguientes métricas de Prometheus graficadas en Grafana:

Prueba de Posts

POSTlocalhost:61577/postPokemonSend

QueryHeadersAuthBodyTestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

Form Encoded

☒ Id001

☒ Namepepe

☒ Type1steel

☒ Type2dragon

☒ Categorybig

☒ Heightftall

☒ Heightmten

Status: 201 CREATEDSize: 41 BytesTime: 302 ms

ResponseHeaders5CookiesResultsDocs

1{

2"message": "Data inserted successfully"

3}







Prueba de Updates

PUTlocalhost:61577/putPokemon/001Send

QueryHeaders2AuthBody1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

Form Encoded

☒ Idfr

☒ Nametilin

☒ Type1as

☒ Type2as

☒ Categoryvf

☒ Heightfvf

☒ Heightmvf

Status: 200 OKSize: 40 BytesTime: 301 ms

ResponseHeaders5CookiesResultsDocs()

1{

2"message": "Data updated successfully"

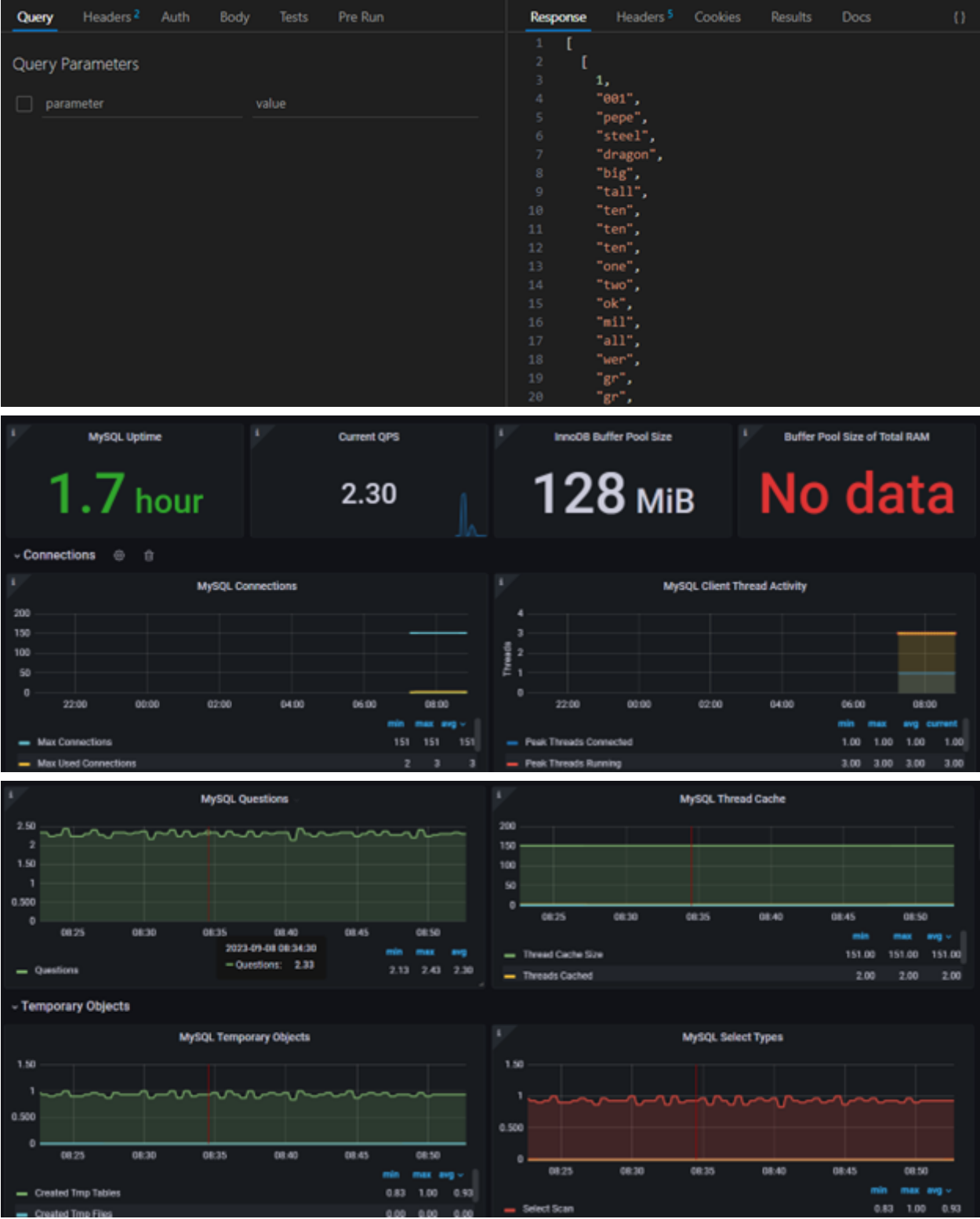
3}







Prueba de Gets



MySQL Uptime

1.7 hour

Current QPS

2.30

InnoDB Buffer Pool Size

128 MiB

Buffer Pool Size of Total RAM

No data

Connections

MySQL Connections

200

150

100

50

0

22:00

00:00

02:00

04:00

06:00

08:00

Min

Max

Avg

151

151

151

Max Connections

Max Used Connections

2

3

3

MySQL Client Thread Activity

4

3

2

1

0

22:00

00:00

02:00

04:00

06:00

08:00

Min

Max

Avg

Current

1.00

1.00

1.00

1.00

Peak Threads Connected

Peak Threads Running

3.00

3.00

3.00

3.00

MySQL Questions

2.50

2.00

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

Min

Max

Avg

2.13

2.43

2.30

Questions

2023-09-08 08:34:30

Questions: 2.33

MySQL Thread Cache

200

150

100

50

0

08:25

08:30

08:35

08:40

08:45

08:50

Min

Max

Avg

151.00

151.00

151.00

Thread Cache Size

Threads Cached

2.00

2.00

2.00

MySQL Temporary Objects

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

Min

Max

Avg

0.83

1.00

0.93

Created Temp Tables

Created Temp Files

0.00

0.00

0.00

MySQL Select Types

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

Min

Max

Avg

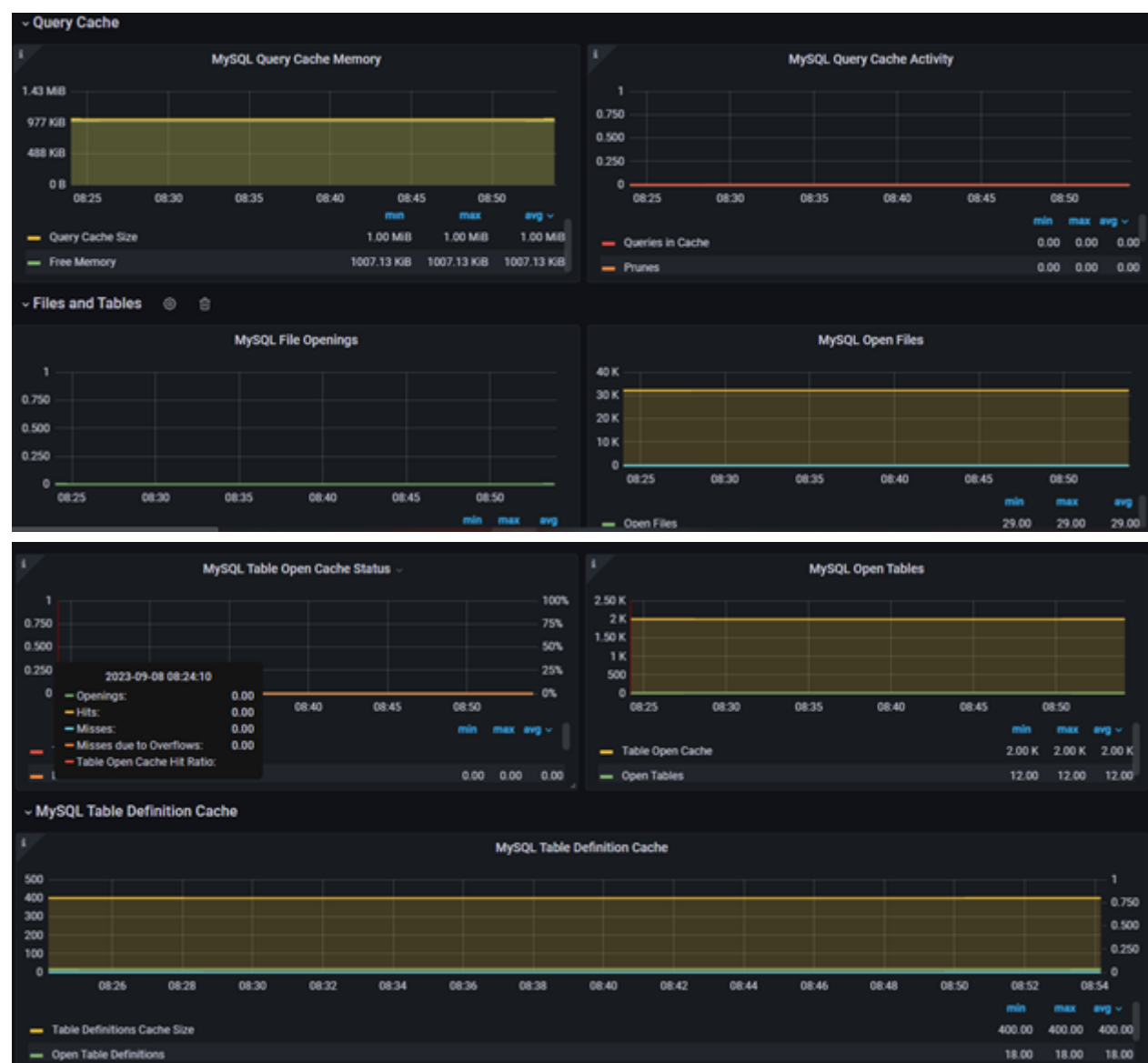
0.83

1.00

0.93

Select Scan





DELETE

localhost:61577/deletePokemon/001

Send

Query

Headers 2

Auth

Body

Tests

Pre Run

Query Parameters

☐

parameter

value

Status: 204 NO CONTENT

Size: 0 Bytes

Time: 303 ms

Response

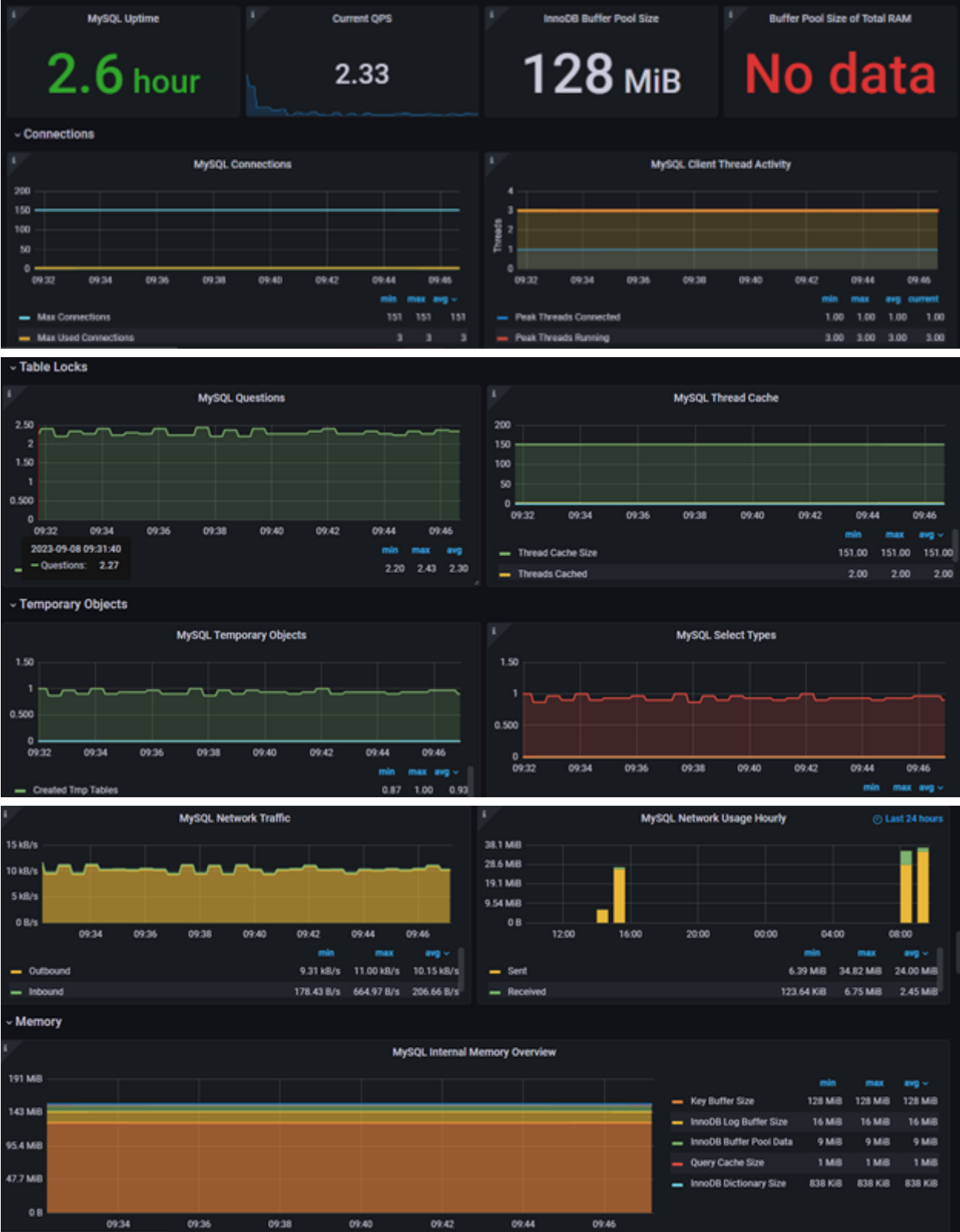
Headers 4

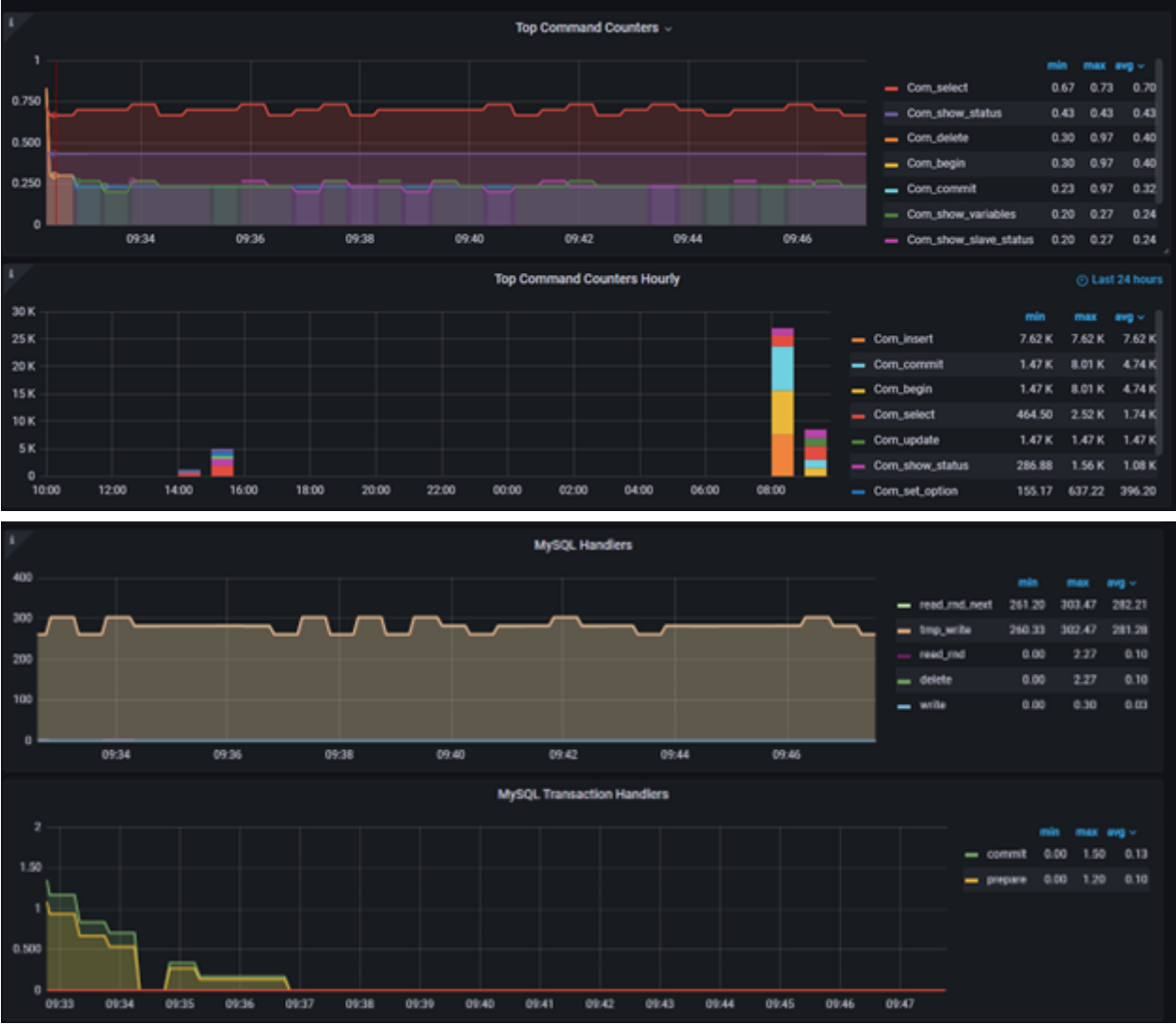
Cookies

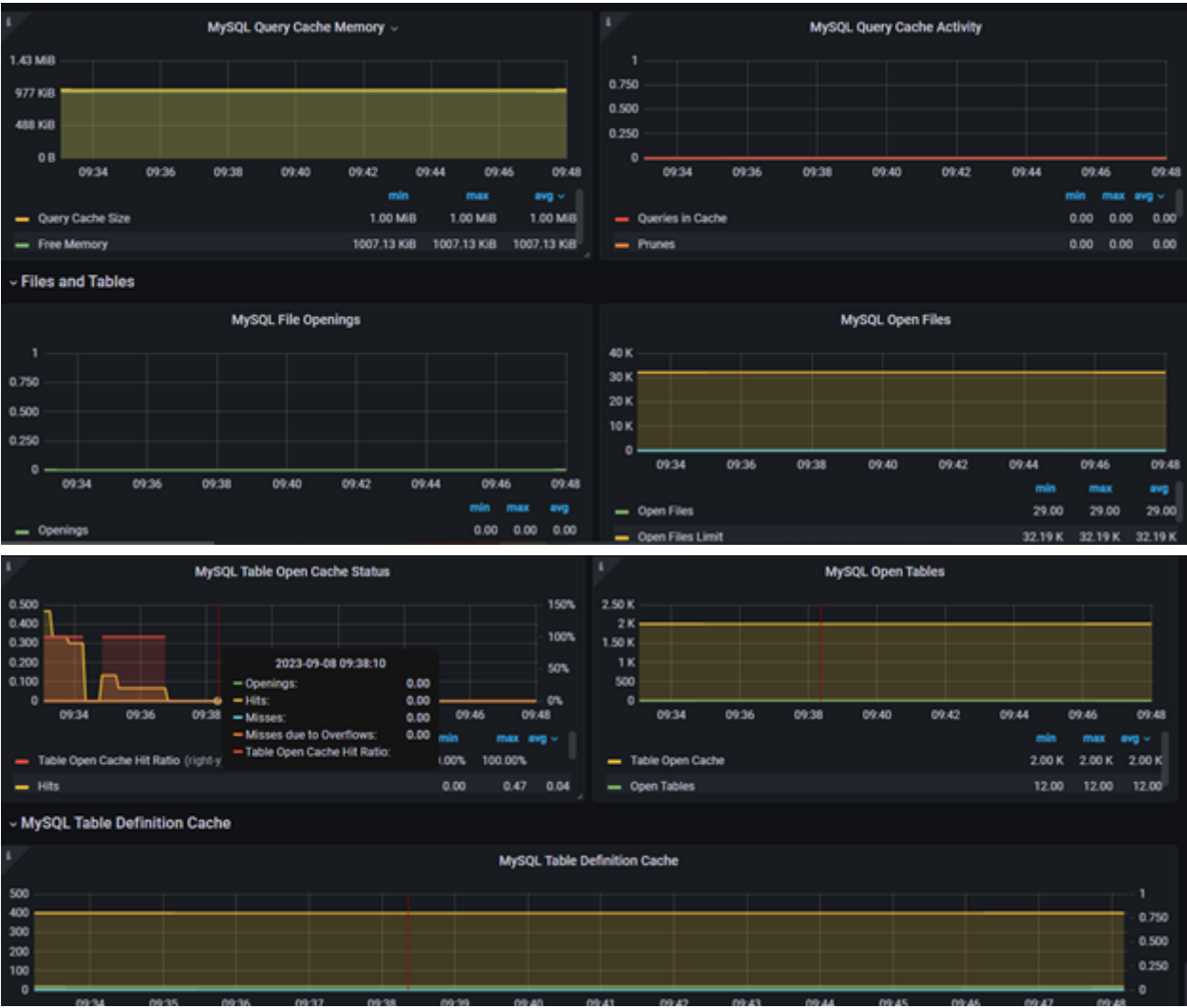
Results

Docs

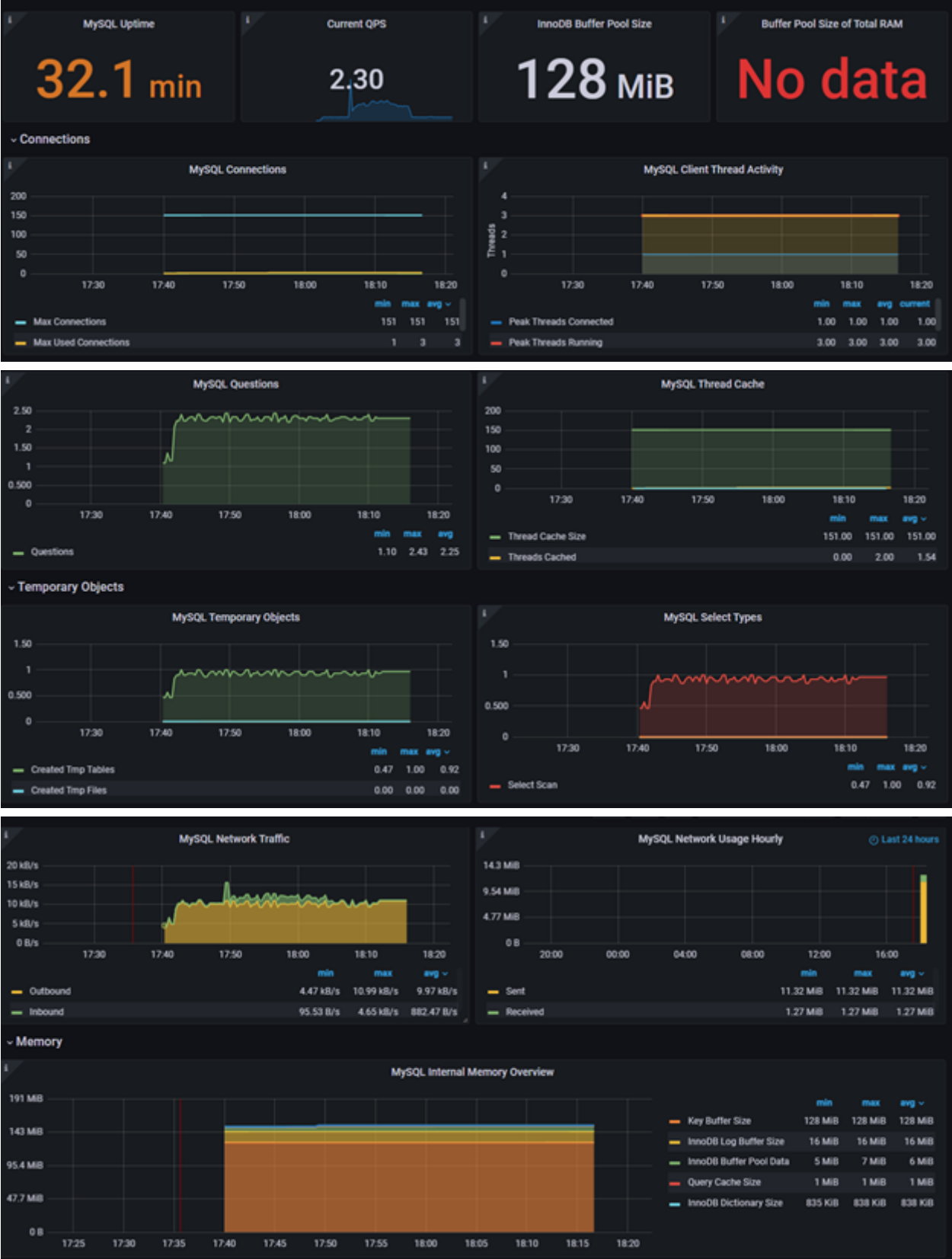
1

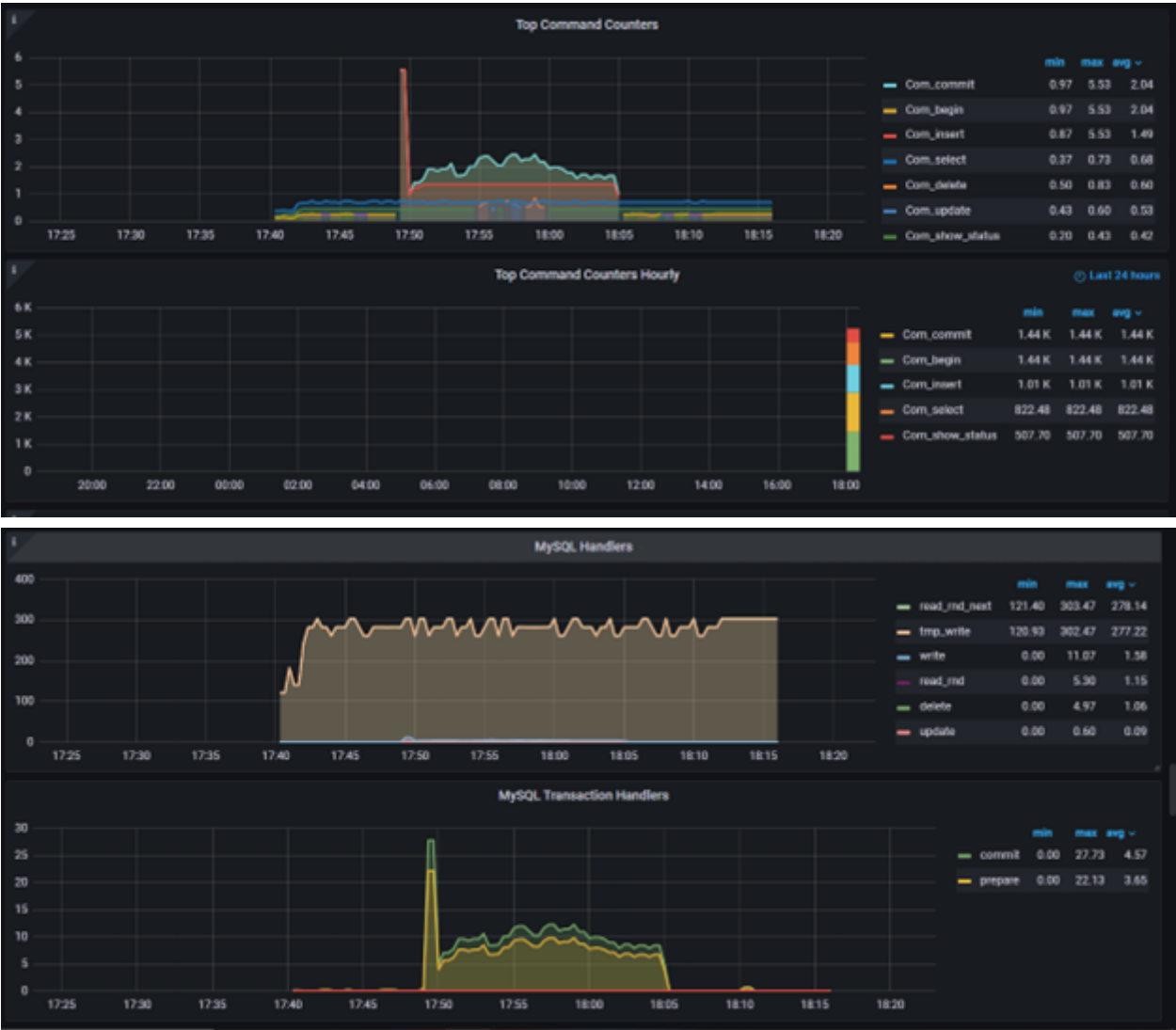






Prueba de Combinaciones







Para la aplicación intermediaria se obtuvieron los siguientes datos:

Prueba de Posts



Prueba de Updates



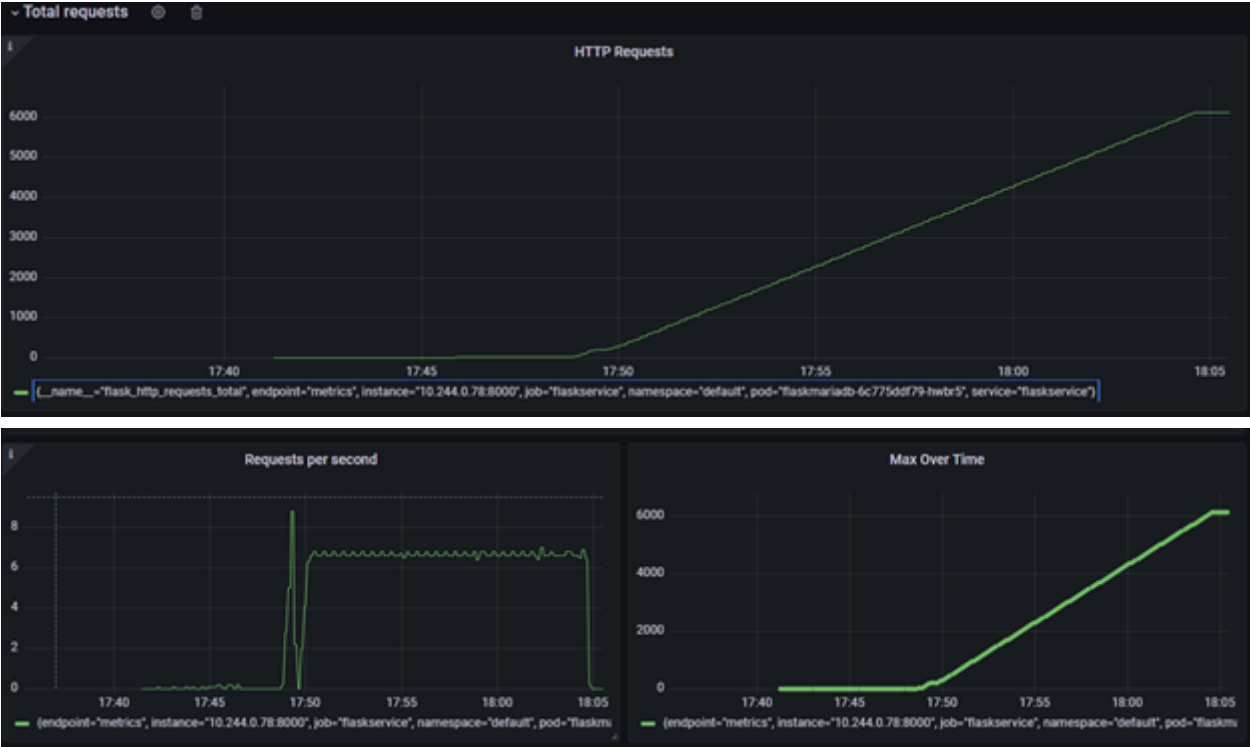
Prueba de Gets



Prueba de Deletes



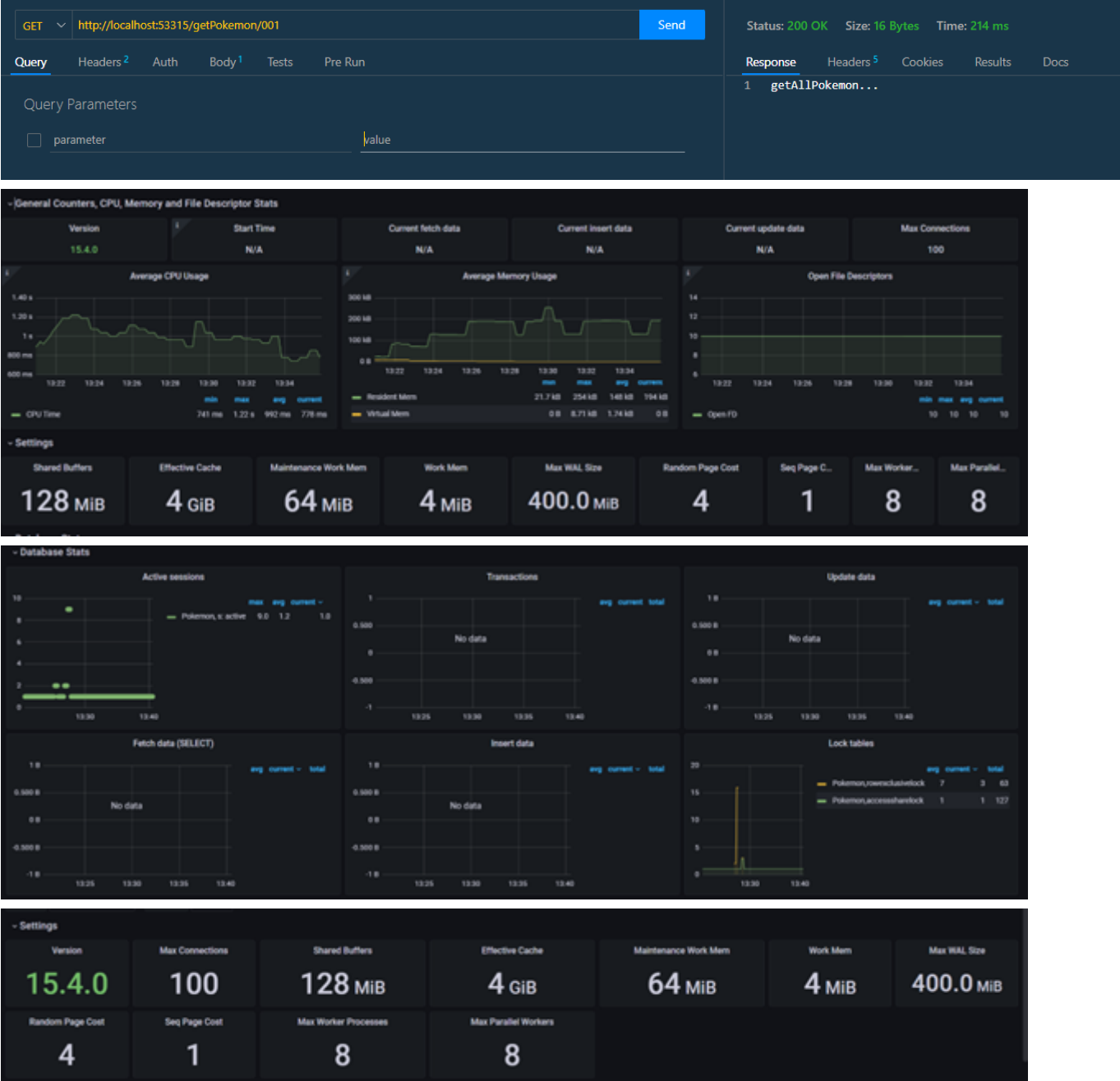
####Prueba de Combinaciones



PostgreSQL

Para la base de datos de PostgreSQL, con las pruebas realizadas se obtuvieron las siguientes métricas de Prometheus graficadas en Grafana:

Prueba de Posts



GET

http://localhost:53315/getAllPokemon

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

Query Parameters

☐

parameter

value

Status: 200 OK

Size: 16 Bytes

Time: 71 ms

Response

Headers⁵

Cookies

Results

Docs

1 getAllPokemon...

POST

http://localhost:53315/postPokemon

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

Form Encoded

☒

Id

001

☒

Name

Bulbasaur

☒

Type1

Grass

☒

Type2

Grass

☒

Category

Seed Pokemon

☒

Heightf

2.04

☒

Heightm

0.7

☒

Weightlbs

15.2

☒

CaptureRate

45

☒

EggSteps

5120

☒

ExpGroup

Medium Slow

Status: 200 OK

Size: 14 Bytes

Time: 141 ms

Response

Headers⁵

Cookies

Results

Docs

1 PostPokemon...

Preview





Prueba de Updates

PUT

http://localhost:53315/putPokemon/001

Send

Status: 200 OK

Size: 21 Bytes

Time: 94 ms

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

Form Encoded

☒ Id

001

☒ Name

Bulbasaur

☒ Type1

Grass

☒ Type2

Grass

☒ Category

Seed Pokemon

☒ Heightf

2.04

☒ Heightm

0.7

☒ Weightlbs

15.2

☒ CaptureRate

45

☒ EggSteps

5120

☒ ExpGroup

Medium Slow

Response

Headers⁵

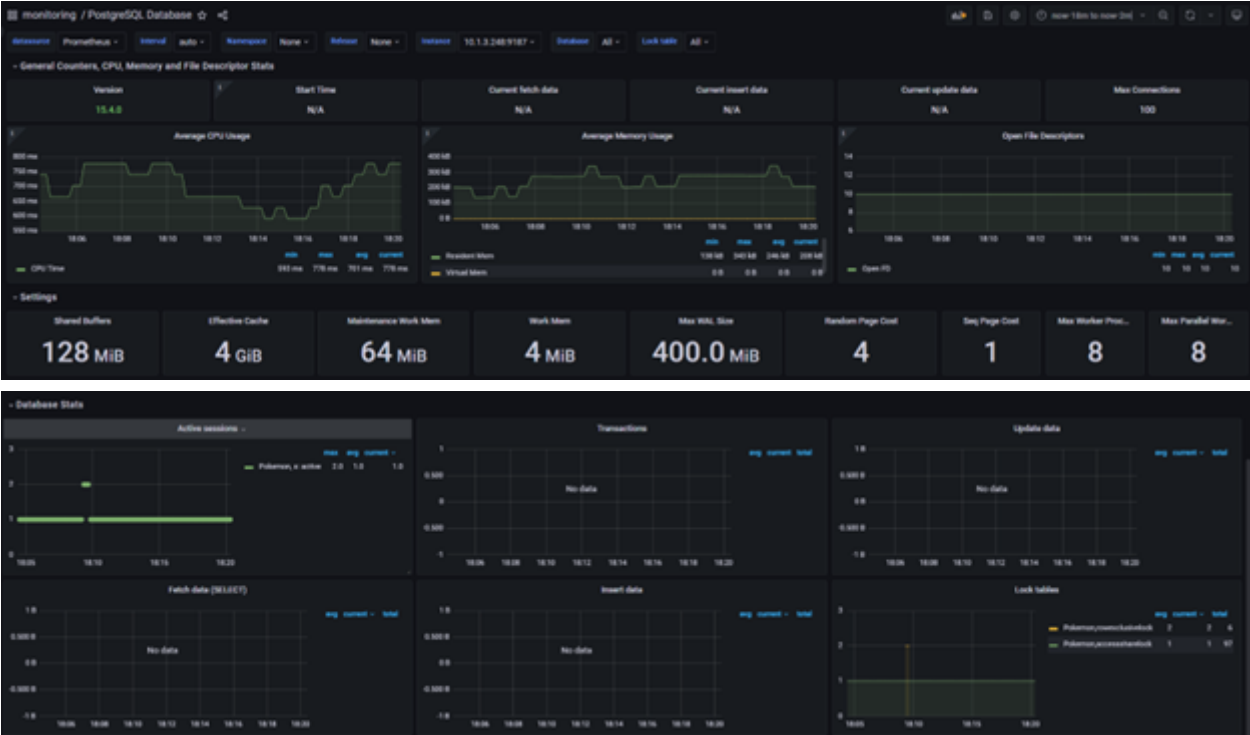
Cookies

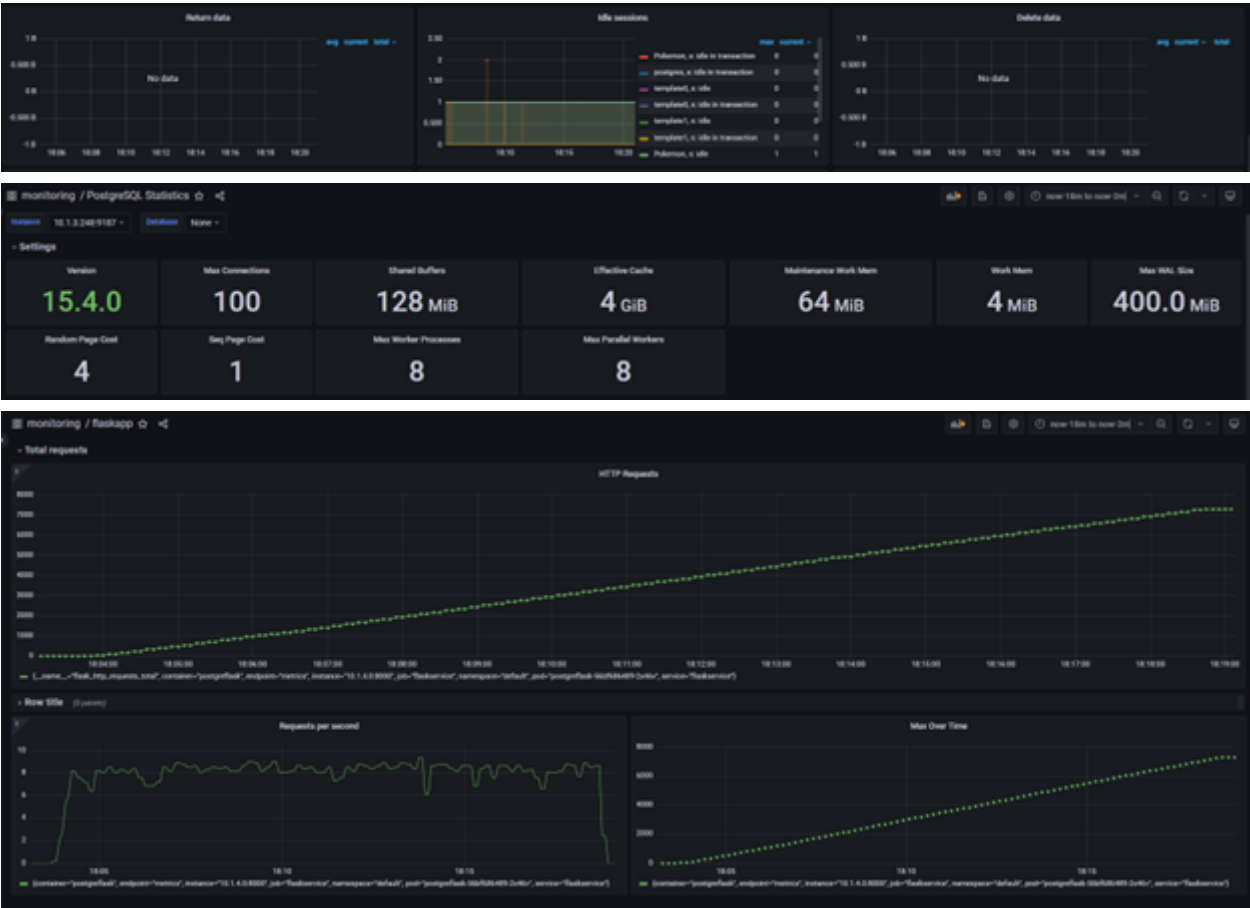
Results

Docs

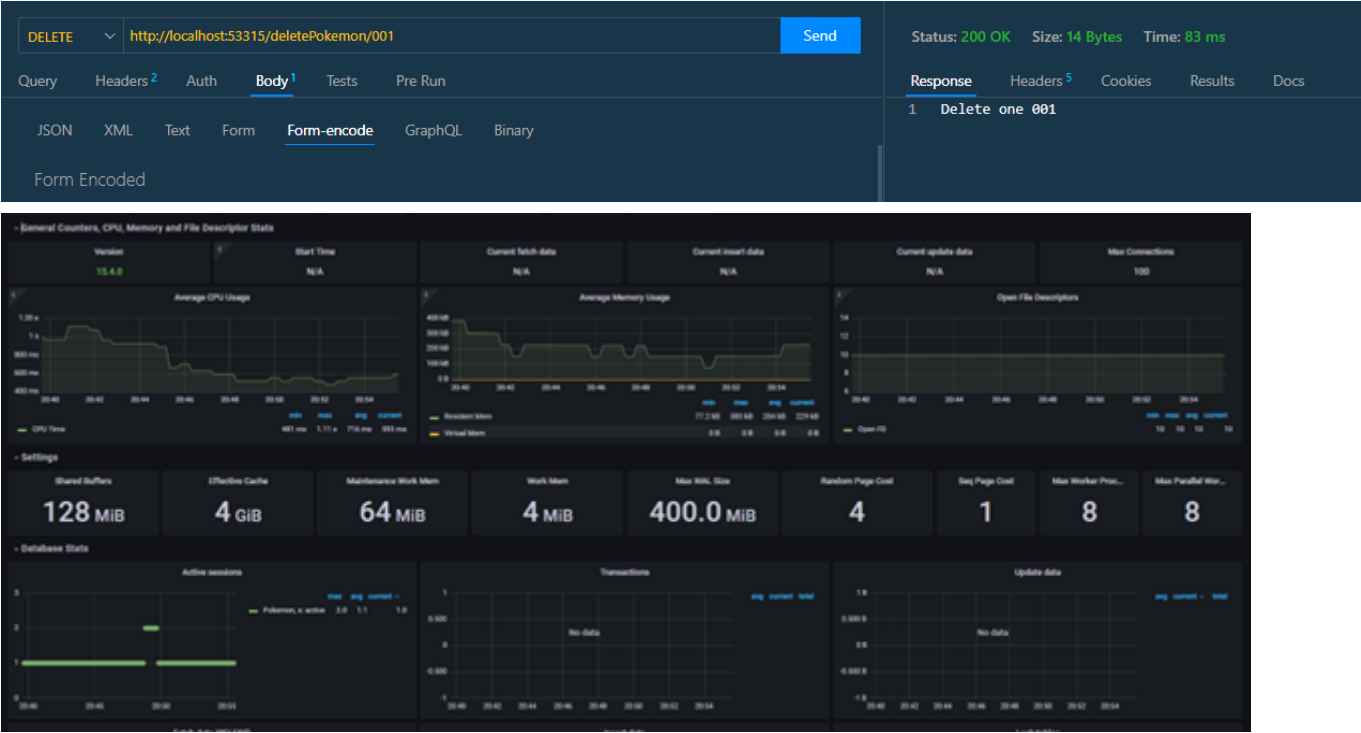
1 putPokemon PostgreSQL

Preview





Prueba de Deletes





Prueba de Combinaciones





PostgreSQL HA

Para la base de datos de PostgreSQL HA, con las pruebas realizadas se obtuvieron las siguientes métricas de de Prometheus graficadas en Grafana:

Prueba de Posts

POSTlocalhost:61577/postPokemonSend

QueryHeadersAuthBodyTestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

Form Encoded

☒ Id001

☒ Namepepe

☒ Type1steel

☒ Type2dragon

☒ Categorybig

☒ Heightftall

☒ Heightmten

Status: 201 CREATEDSize: 41 BytesTime: 302 ms

ResponseHeadersCookiesResultsDocs

1 {

2 "message": "Data inserted successfully"

3 }







Prueba de Updates

PUTlocalhost:61577/putPokemon/001Send

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

Form Encoded

☒ Idfr

☒ Nametilin

☒ Type1as

☒ Type2as

☒ Categoryvf

☒ Heightfvf

☒ Heightmvf

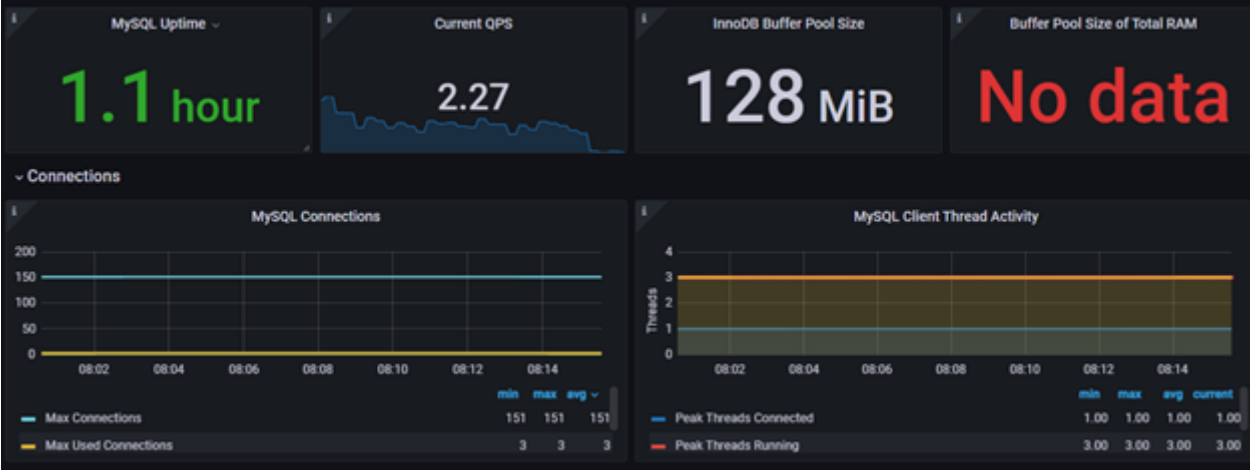
Status: 200 OKSize: 40 BytesTime: 301 ms

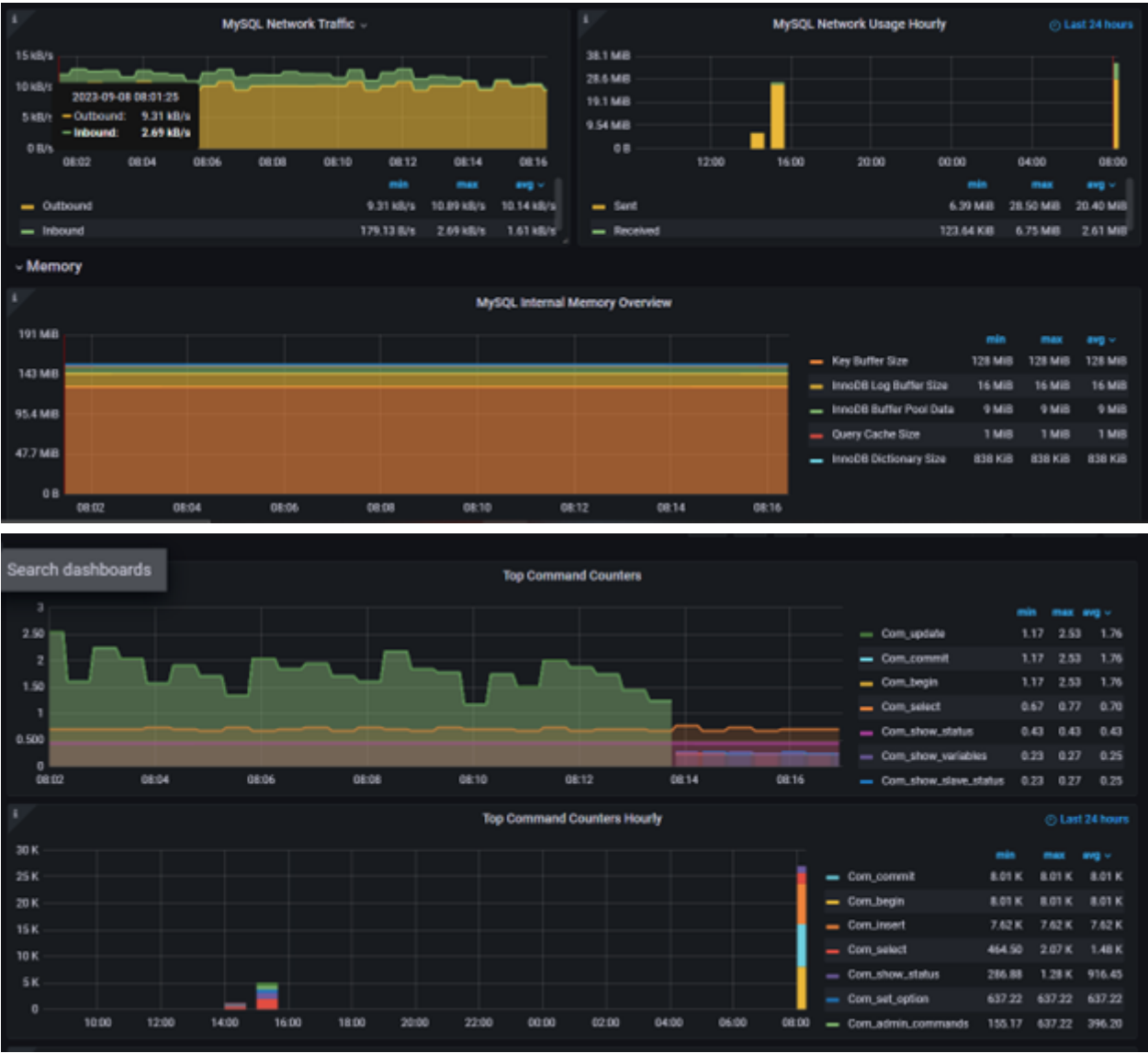
ResponseHeaders 5CookiesResultsDocs {}

1 {

2 "message": "Data updated successfully"

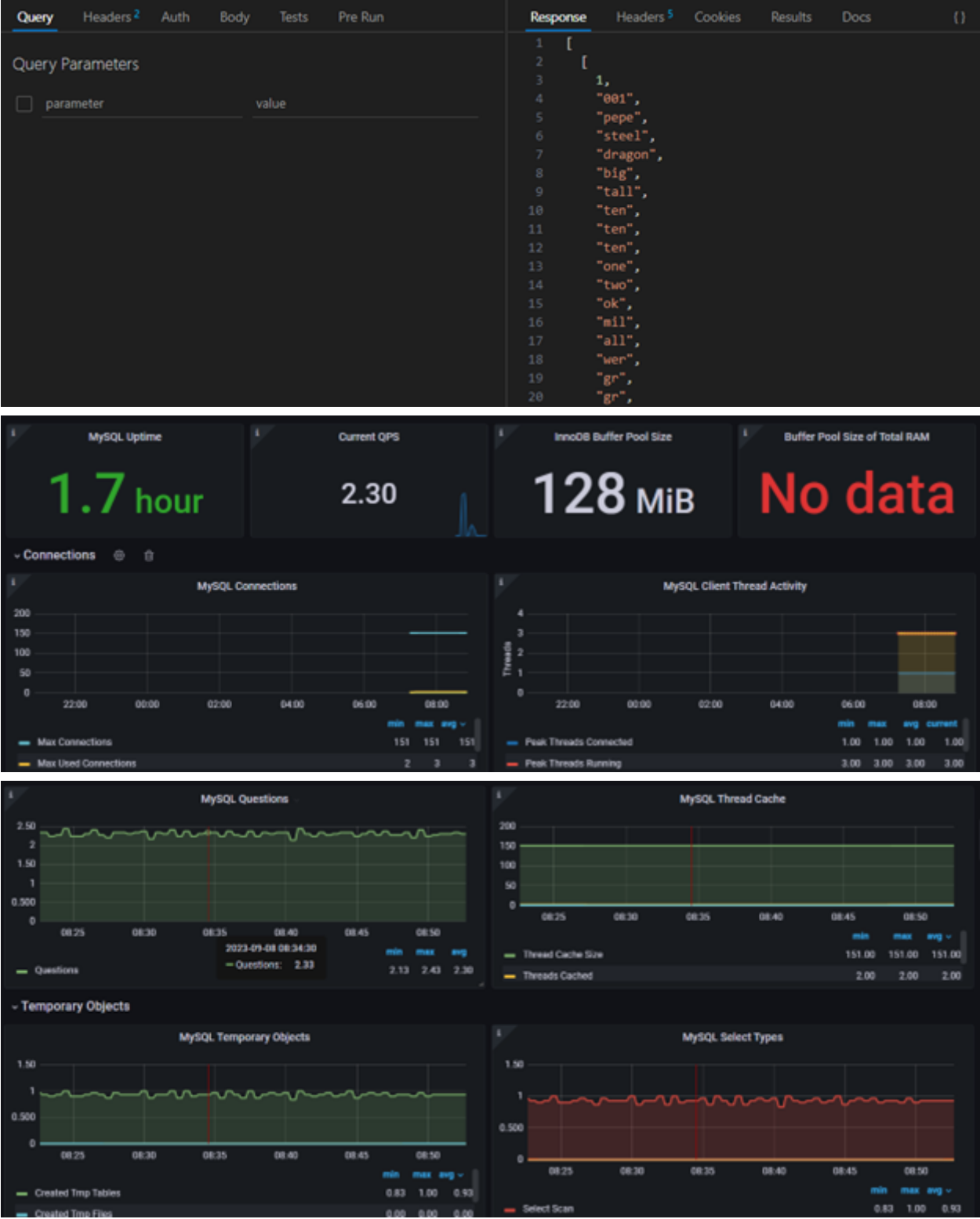
3 }







Prueba de Gets



MySQL Uptime

1.7 hour

Current QPS

2.30

InnoDB Buffer Pool Size

128 MiB

Buffer Pool Size of Total RAM

No data

Connections

MySQL Connections

200

150

100

50

0

22:00

00:00

02:00

04:00

06:00

08:00

min

max

avg

151

151

151

Max Connections

Max Used Connections

2

3

3

MySQL Client Thread Activity

Threads

4

3

2

1

0

22:00

00:00

02:00

04:00

06:00

08:00

min

max

avg

current

1.00

1.00

1.00

1.00

Peak Threads Connected

Peak Threads Running

3.00

3.00

3.00

3.00

MySQL Questions

2.50

2.00

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

min

max

avg

2.13

2.43

2.30

Questions

2023-09-08 08:34:30

Questions: 2.33

MySQL Thread Cache

200

150

100

50

0

08:25

08:30

08:35

08:40

08:45

08:50

min

max

avg

151.00

151.00

151.00

Thread Cache Size

Threads Cached

2.00

2.00

2.00

Temporary Objects

MySQL Temporary Objects

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

min

max

avg

0.83

1.00

0.93

Created Temp Tables

Created Temp Files

0.00

0.00

0.00

MySQL Select Types

1.50

1.00

0.500

0

08:25

08:30

08:35

08:40

08:45

08:50

min

max

avg

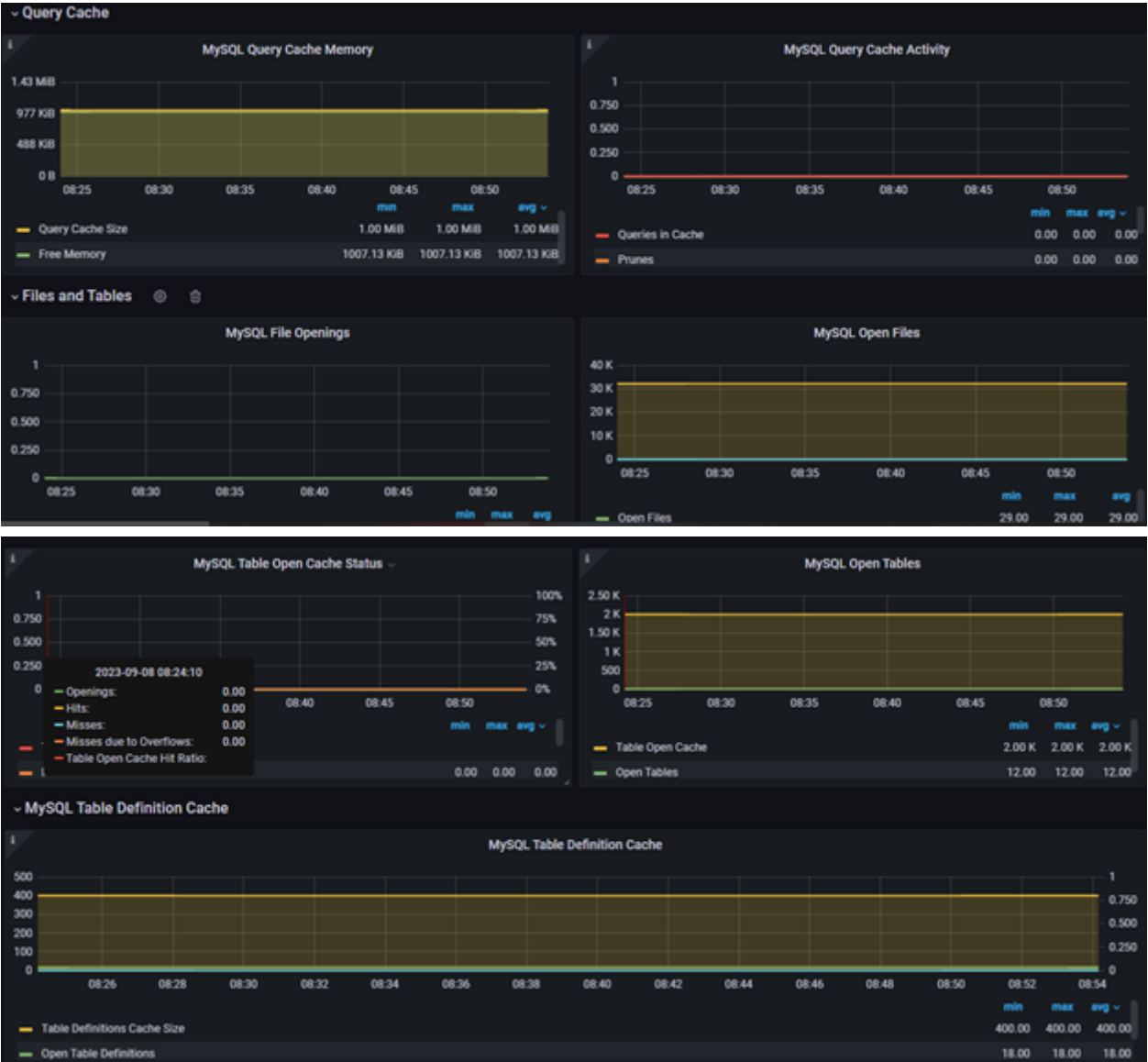
0.83

1.00

0.93

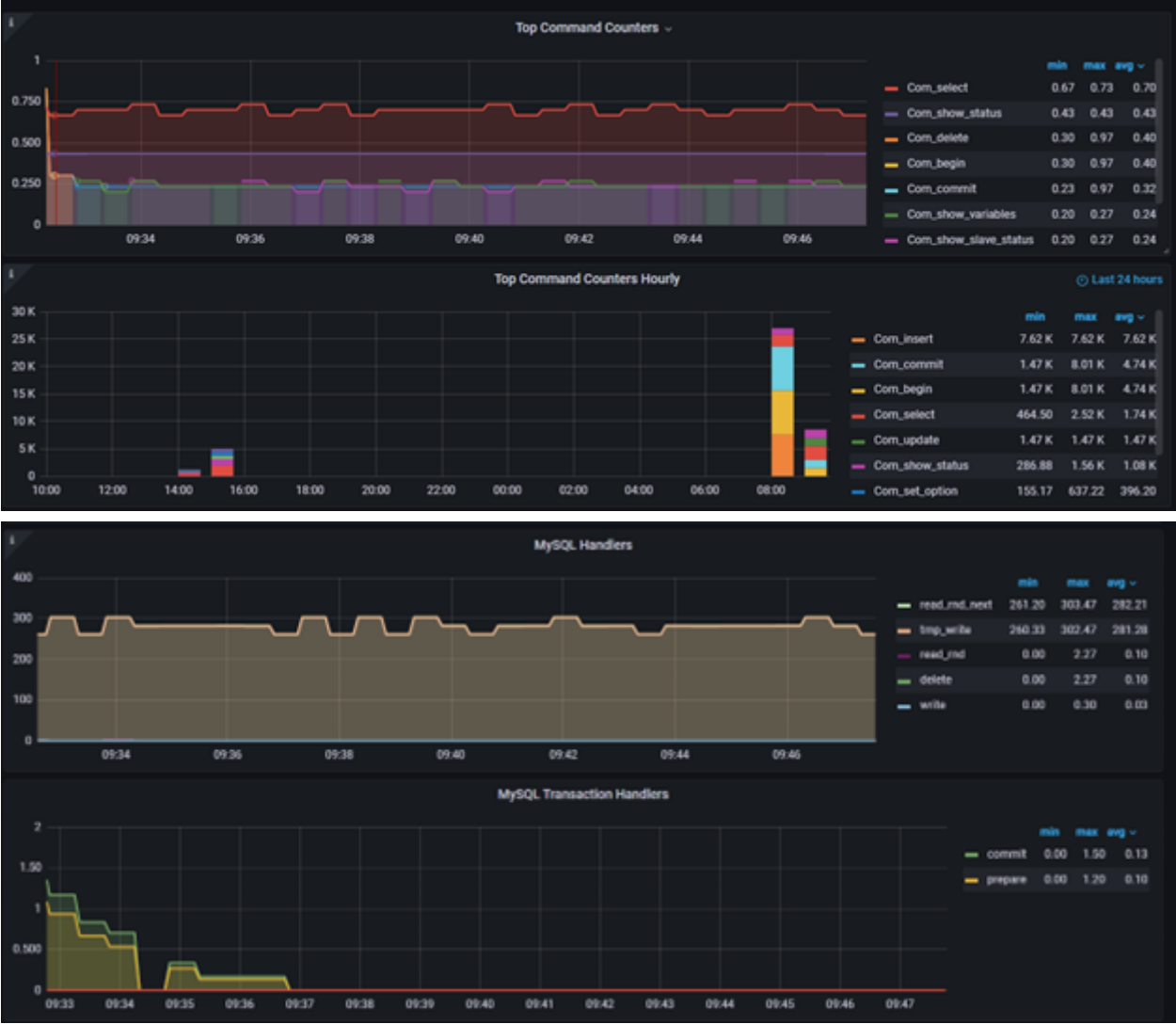
Select Scan

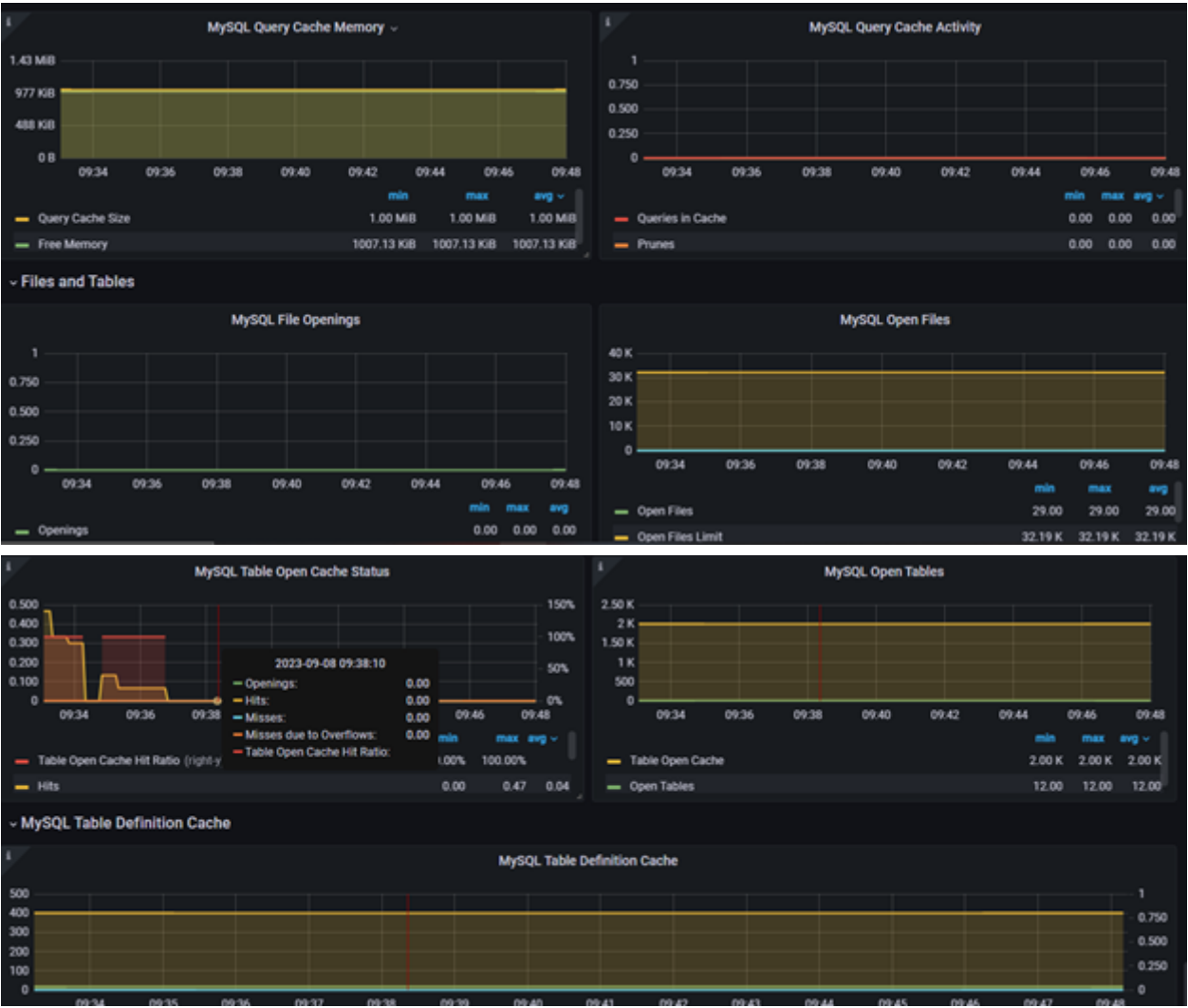




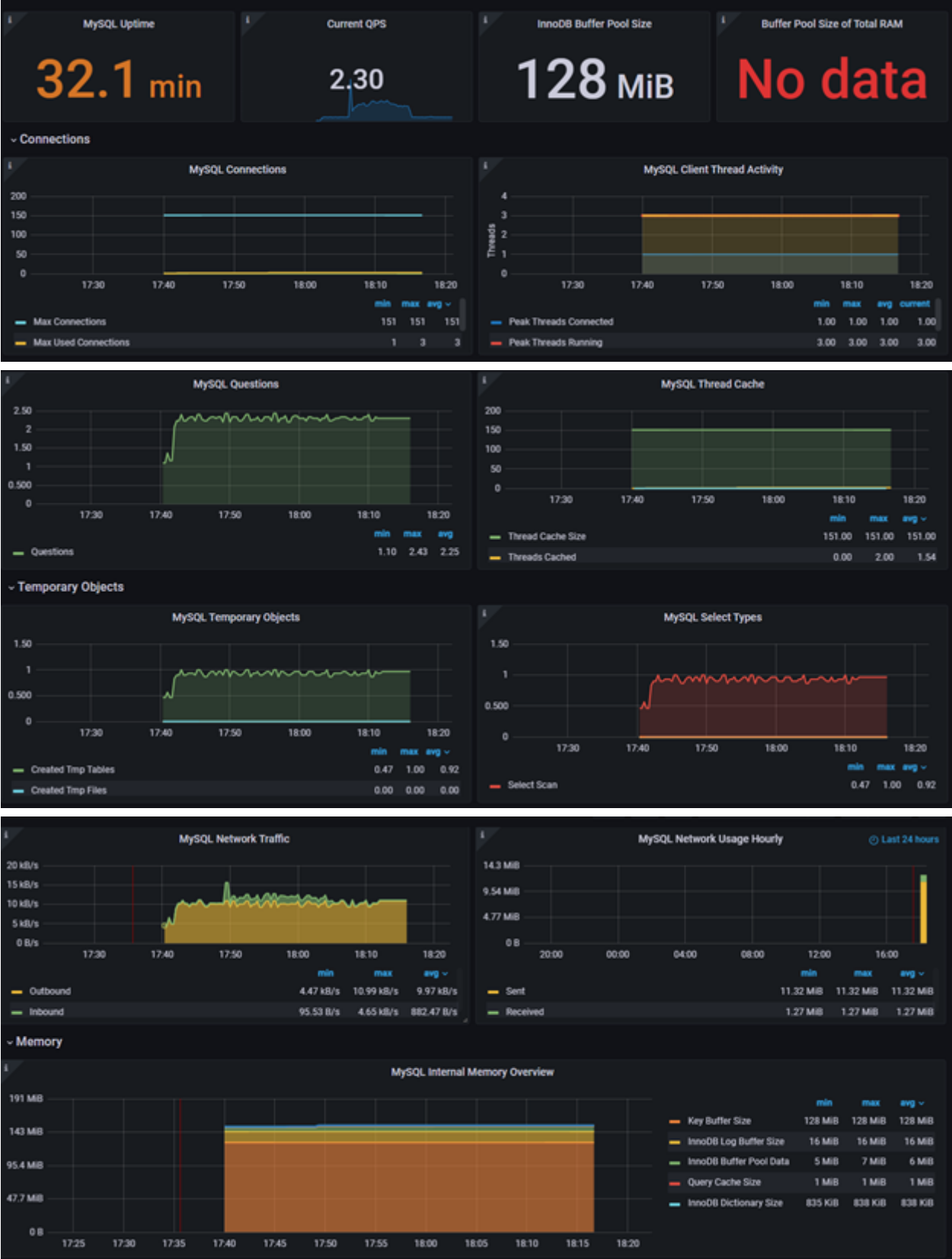
Prueba de Deletes

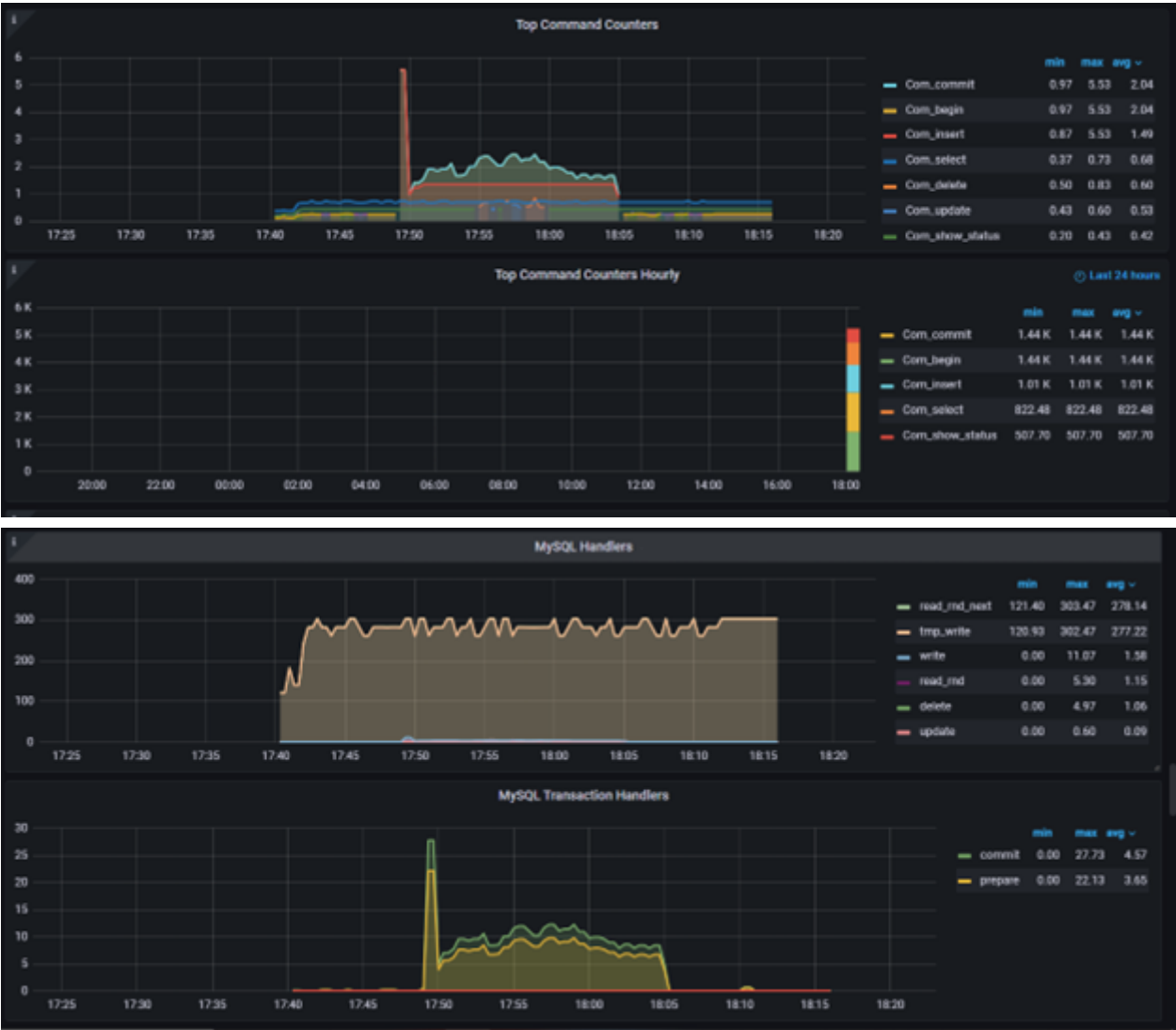






Prueba de Combinaciones







Para la

aplicación intermediaria se obtuvieron los siguientes datos:

Prueba de Posts



Prueba de Updates



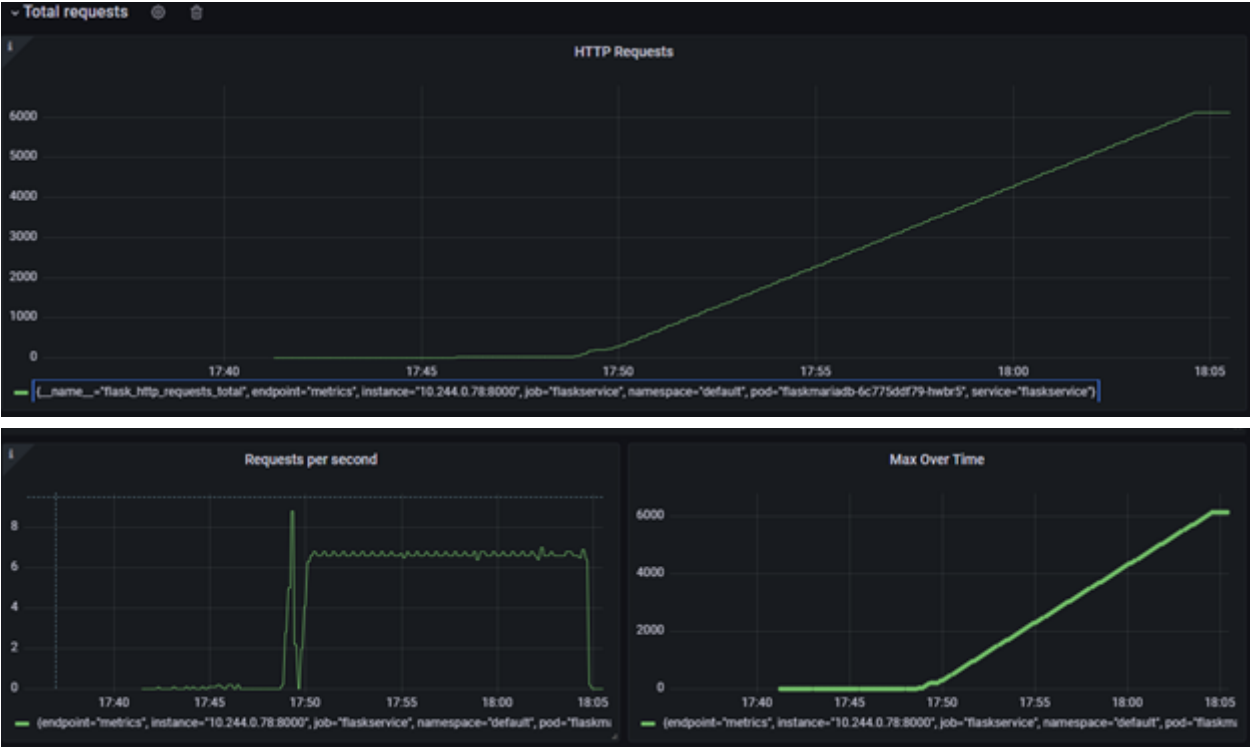
Prueba de Gets



Prueba de Deletes



Prueba de Combinaciones



MongoDB

Para la base de datos de MongoDB, con las pruebas realizadas se obtuvieron las siguientes métricas de de Prometheus graficadas en Grafana:

Prueba de Posts

POST

http://localhost:53347/postPokemon

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

☒

Id

001

☒

Name

Bulbasaur

☒

Type1

Grass

☒

Type2

Poison

☒

Category

Seed Pokémon

☒

Heightf

2'

☒

Heightm

0.7

☒

Weightlbs

15.2

☒

Weightkg

6.9

☒

CaptureRate

45

☒

EggSteps

5120

☒

ExpGroup

Slow

Status: 200 OK

Size: 402 Bytes

Time: 152 ms

Response

Headers 5

Cookies

Results

Docs

{}

≡

1

Pokemon {'Id': '001', 'Name': 'Bulbasaur', 'Type1': 'Grass', 'Type2': 'Poison', 'Category': 'Seed Pokémon', 'Heightf': '2'', 'Heightm': '0.7', 'Weightlbs': '15.2', 'Weightkg': '6.9', 'CaptureRate': '45', 'EggSteps': '5120', 'ExpGroup': 'Slow', 'Total': '318', 'HP': '45', 'Attack': '49', 'Defense': '49', 'SpAttack': '65', 'SpDefense': '65', 'Speed': '45', '_id': ObjectId('64f9e71c9de33fea3fcc534')}

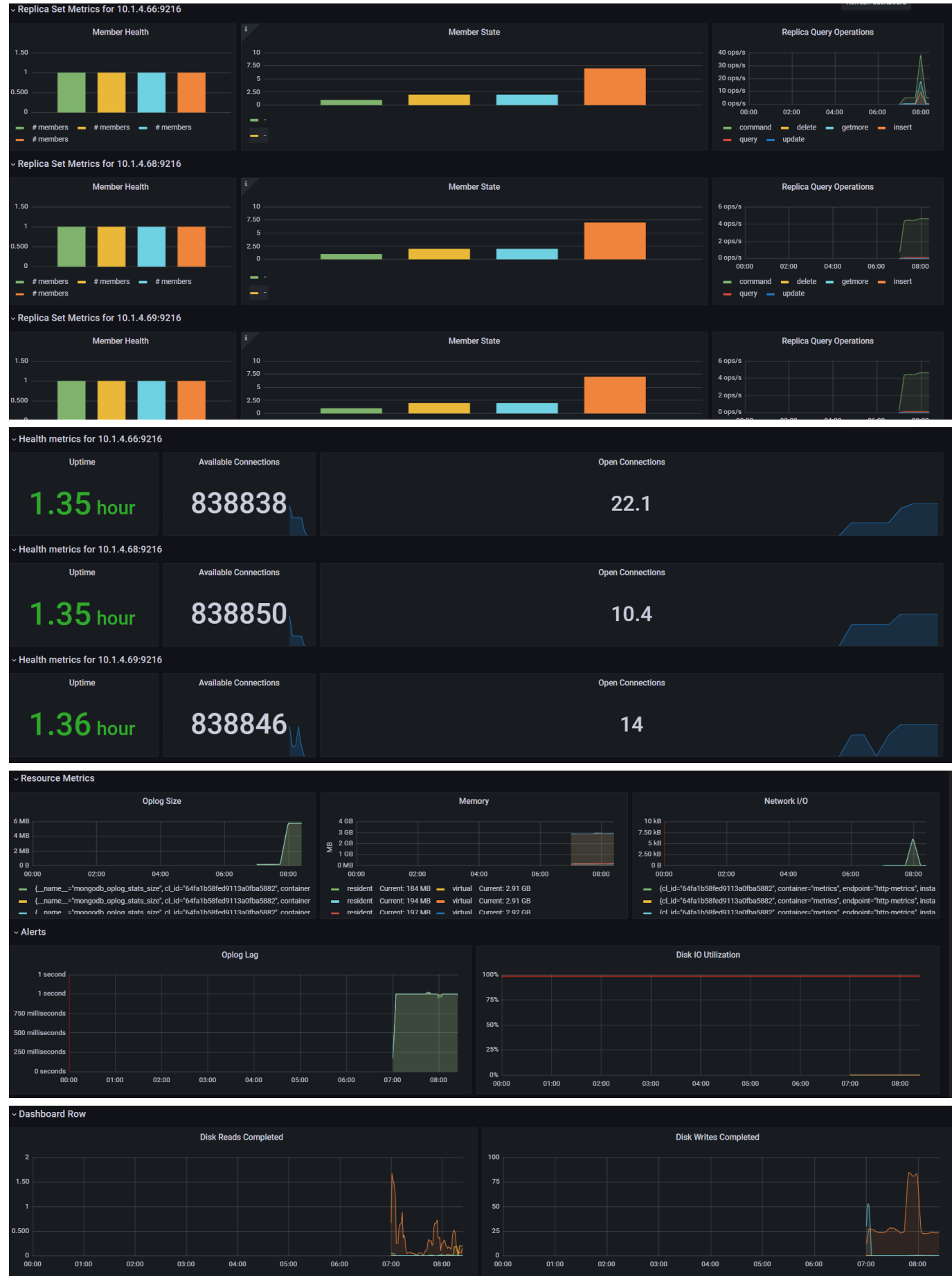
2

3

4

5





Prueba de Updates

PUT

http://localhost:53347/putPokemon/001

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

☒

Id

001

☒

Name

Bulbasaur

☒

Type1

Grass

☒

Type2

Poison

☒

Category

Seed Pokémon

☒

Heightf

2'

☒

Heightm

0.7

Response

Headers⁵

Cookies

Results

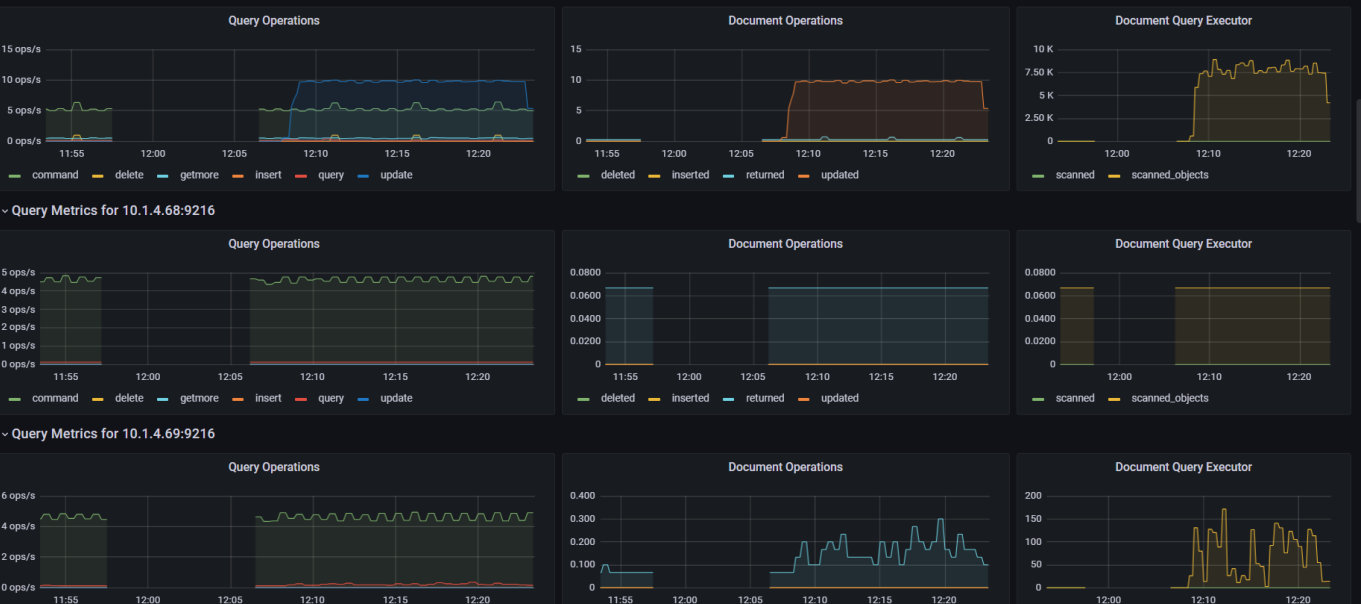
Docs

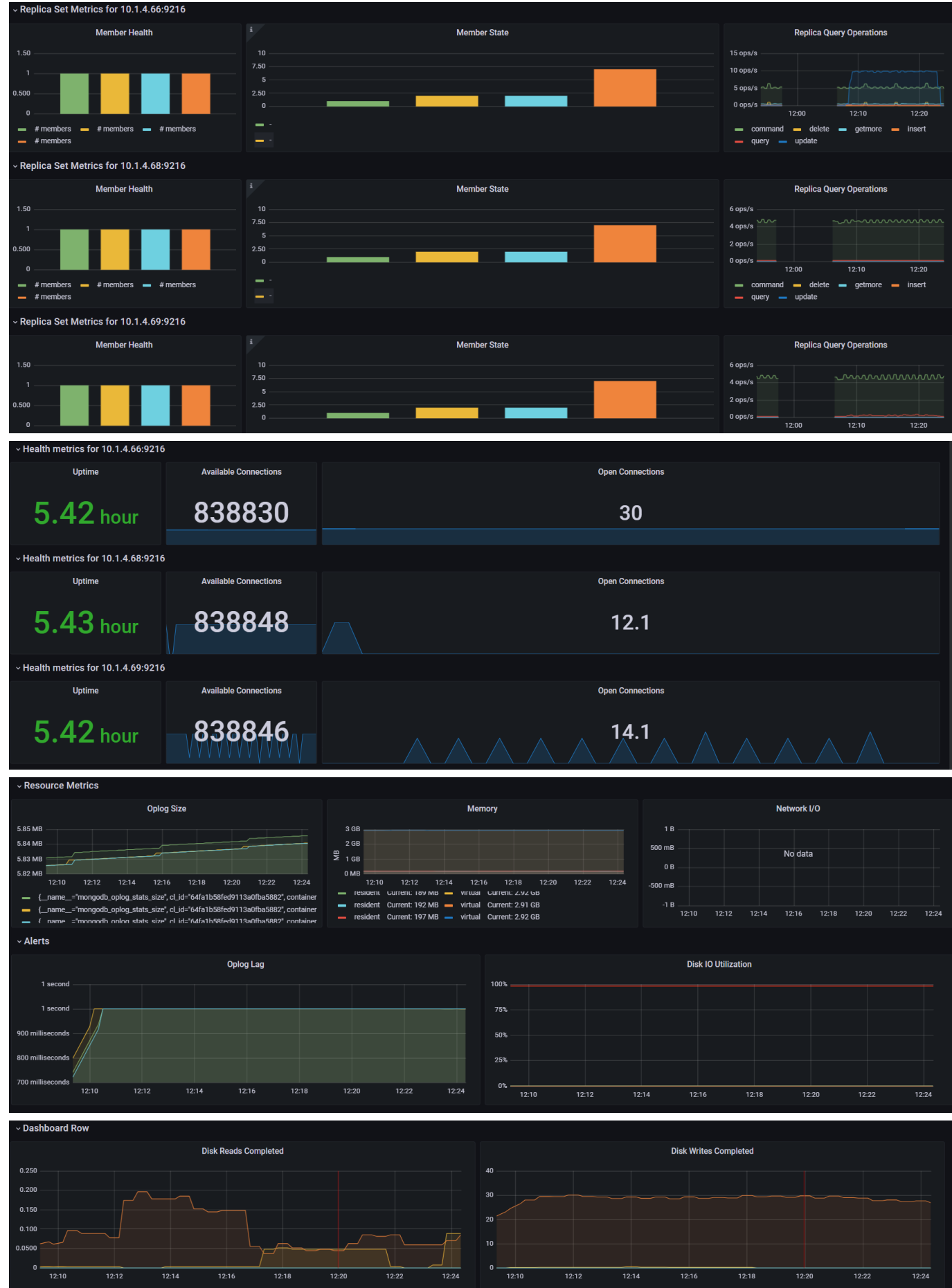
{}

≡

1

Pokemon Update {'Id': '001', 'Name': 'Bulbasaur', 'Type1': 'Grass', 'Type2': 'Poison', 'Category': 'Seed Pokémon', 'Heightf': '2', 'Heightm': '0.7', 'Weightlbs': '15.2', 'Weightkg': '6.9', 'CaptureRate': '45', 'EggSteps': '5120', 'ExpGroup': 'Slow', 'Total': '318', 'HP': '45', 'Attack': '49', 'Defense': '49', 'SpAttack': '65', 'SpDefense': '65', 'Speed': '45'}





GET ☐ http://localhost:53347/getPokemon/001

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form **Form-encode** GraphQL Binary

☒	Id	001
☒	Name	Bulbasaur
☒	Type1	Grass
☒	Type2	Poison

Status: 200 OK Size: 402 Bytes Time: 28 ms

Response Headers ⁵ Cookies Results Docs {} ≡

```
1 Get one {'_id': ObjectId('64fb74cc9de33fea3fccc115'), 'Id': '001', 'Name': 'Bulbasaur', 'Type1': 'Grass', 'Type2': 'Poison', 'Category': 'Seed Pokémon', 'Heightf': '2', 'Heightm': '0.7', 'Weightlbs': '15.2', 'Weightkg': '6.9', 'CaptureRate': '45', 'EggSteps': '5120', 'ExpGroup': 'Slow', 'Total': '318', 'HP': '45', 'Attack': '49', 'Defense': '49', 'SpAttack': '65', 'SpDefense': '65', 'Speed': '45'}
```

GET ☐ http://localhost:53347/getAllPokemon

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form **Form-encode** GraphQL Binary

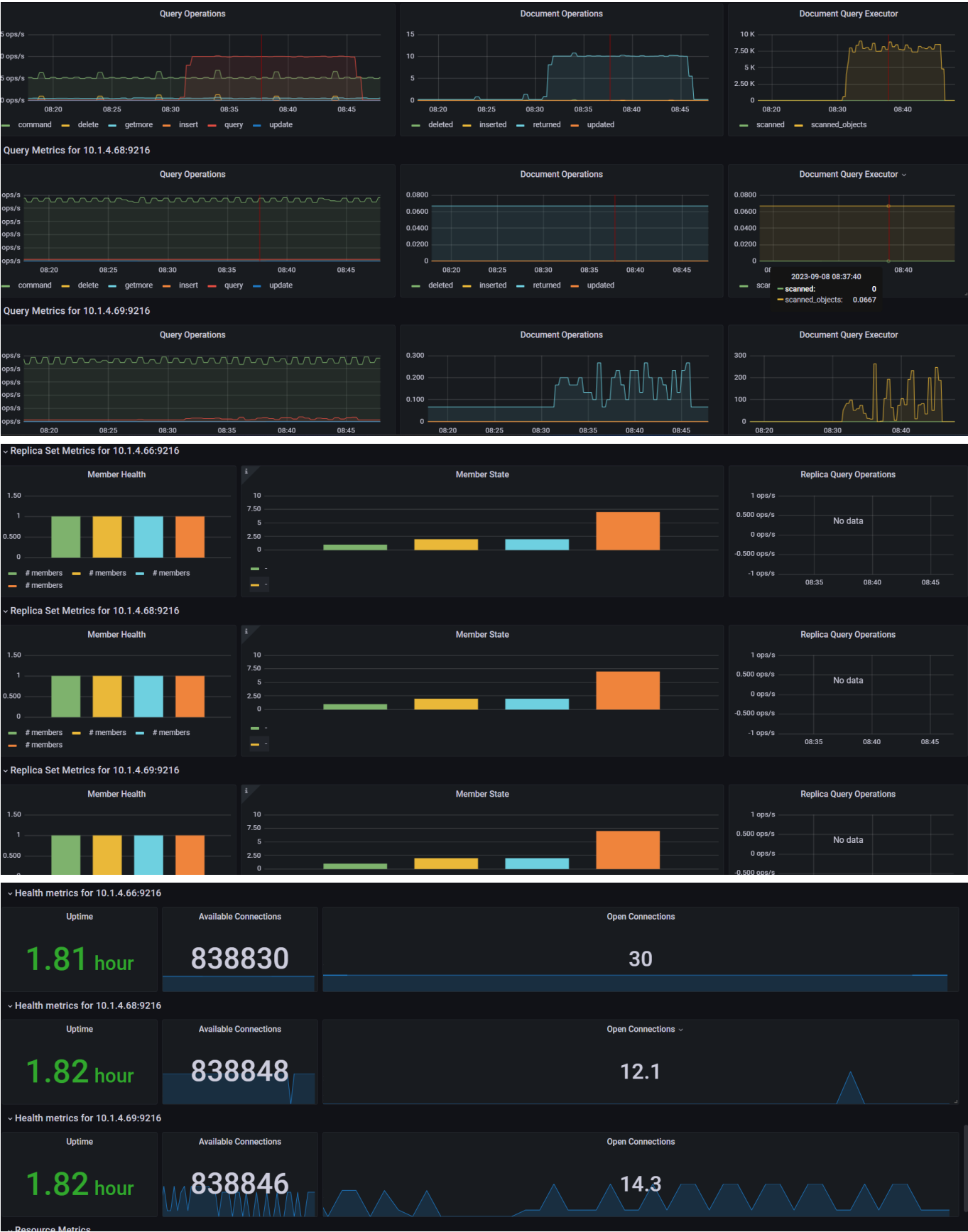
☒	Id	001
☒	Name	Bulbasaur
☒	Type1	Grass
☒	Type2	Poison
☒	Category	Seed Pokémon
☒	Heightf	2
☒	Heightm	0.7
☒	Weightlbs	15.2
☒	Weightkg	6.9
☒	CaptureRate	45
☒	EggSteps	5120
☒	ExpGroup	Slow

Status: 200 OK Size: 699.96 KB Time: 664 ms

Response Headers ⁵ Cookies Results Docs {} ≡

```
1 Pokemones Get All [{'Id': '494', 'Name': 'Victini', 'Type1': 'Psychic', 'Type2': 'Fire', 'Category': 'Victory Pokémon', 'Heightf': '1\04"', 'Heightm': '0.4', 'Weightlbs': '8.8', 'Weightkg': '4', 'CaptureRate': '3', 'EggSteps': '30720', 'ExpGroup': 'Slow', 'Total': '600', 'HP': '100', 'Attack': '100', 'Defense': '100', 'SpAttack': '100', 'SpDefense': '100', 'Speed': '100'}, {'Id': '494', 'Name': 'Victini', 'Type1': 'Psychic', 'Type2': 'Fire', 'Category': 'Victory Pokémon', 'Heightf': '1\04"', 'Heightm': '0.4', 'Weightlbs': '8.8', 'Weightkg': '4', 'CaptureRate': '3', 'EggSteps': '30720', 'ExpGroup': 'Slow', 'Total': '600', 'HP': '100', 'Attack': '100', 'Defense': '100', 'SpAttack': '100', 'SpDefense': '100', 'Speed': '100'}, {'Id': '045', 'Name': 'Vileplume', 'Type1': 'Grass', 'Type2': 'Poison', 'Category': 'Flower Pokémon', 'Heightf': '3\11"', 'Heightm': '1.2', 'Weightlbs': '41.0', 'Weightkg': '18.6', 'CaptureRate': '45', 'EggSteps': '5120', 'ExpGroup': 'Medium Slow', 'Total': '490', 'HP': '75', 'Attack': '80', 'Defense': '85', 'SpAttack': '110'}
```





Query Metrics for 10.1.4.69:9216

Query Operations

Document Operations

Document Query Executor

Replica Set Metrics for 10.1.4.66:9216

Member Health

Member State

Replica Query Operations

Replica Set Metrics for 10.1.4.68:9216

Member Health

Member State

Replica Query Operations

Replica Set Metrics for 10.1.4.69:9216

Member Health

Member State

Replica Query Operations

Health metrics for 10.1.4.66:9216

Uptime

1.81 hour

Available Connections

838830

Open Connections

30

Health metrics for 10.1.4.68:9216

Uptime

1.82 hour

Available Connections

838848

Open Connections

12.1

Health metrics for 10.1.4.69:9216

Uptime

1.82 hour

Available Connections

838846

Open Connections

14.3

Resource Metrics





Member Health

Member State

Replica Query Operations

Health metrics for 10.1.4.66:9216

Uptime

5.83 hour

Available Connections

838824

Open Connections

35.9

Health metrics for 10.1.4.68:9216

Uptime

5.83 hour

Available Connections

838848

Open Connections

12.2

Health metrics for 10.1.4.69:9216

Uptime

5.83 hour

Available Connections

838845

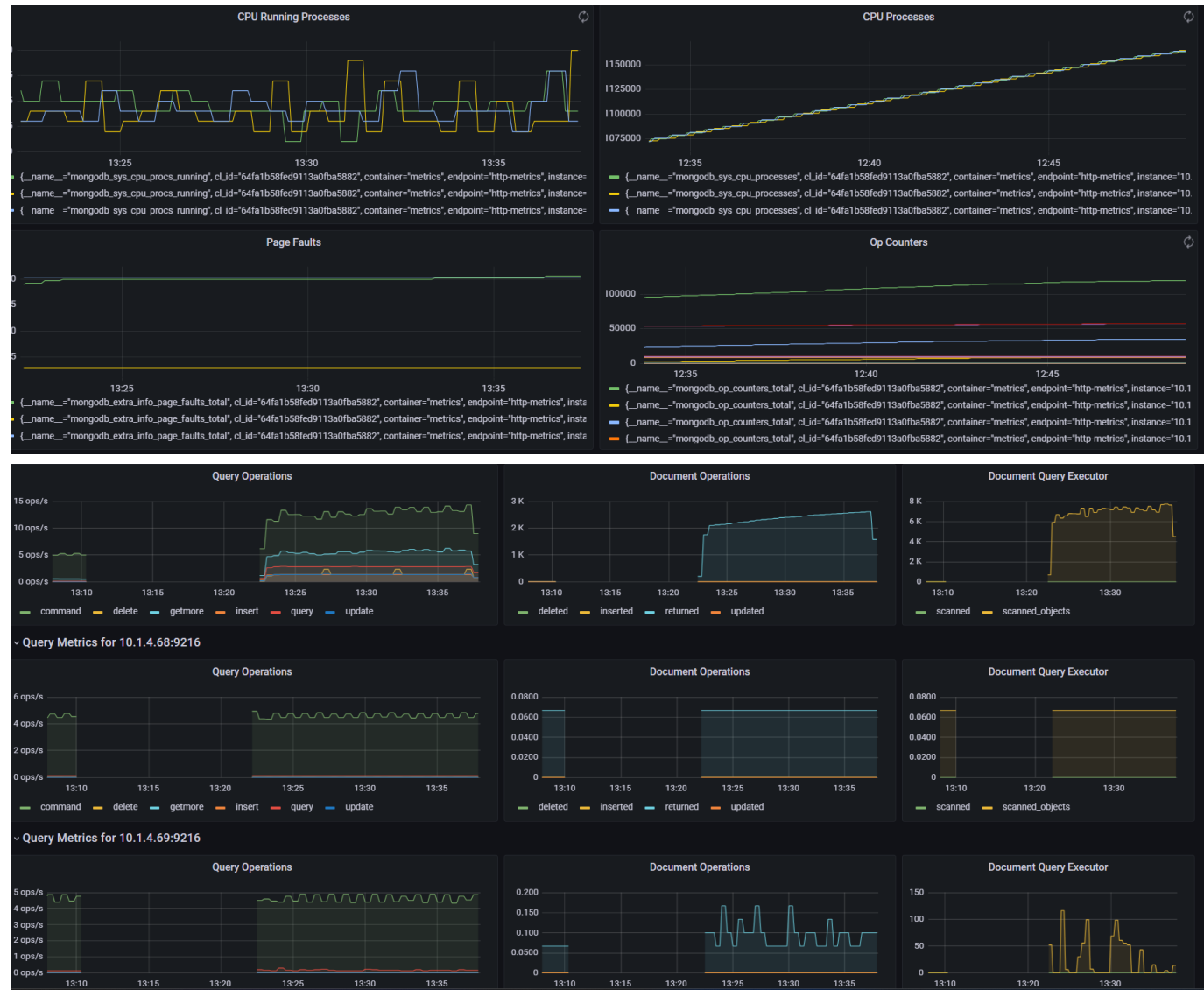
Open Connections

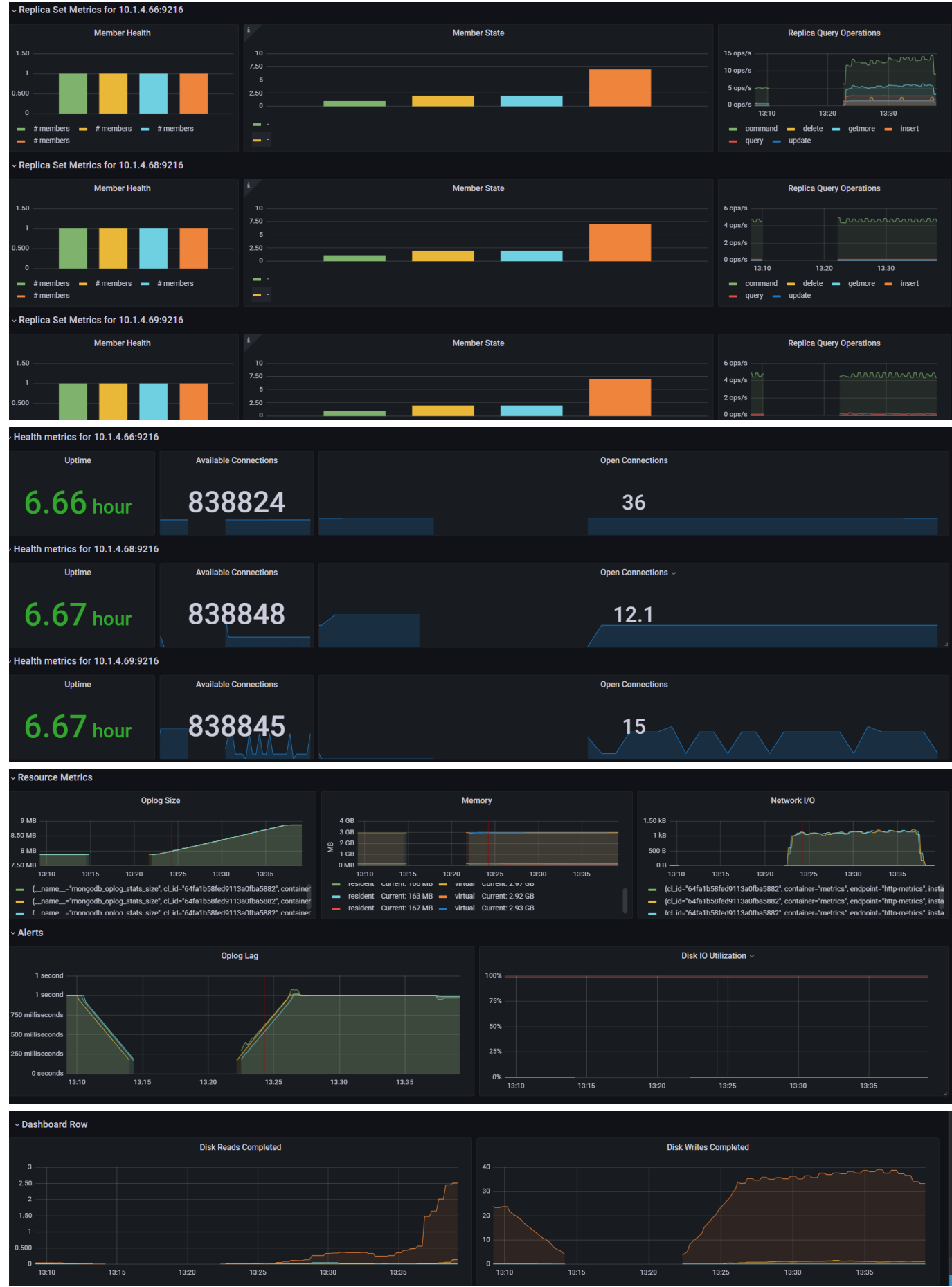
14.9

Resource Metrics



Prueba de Combinaciones







Prueba de Updates



Prueba de Gets



Prueba de Deletes



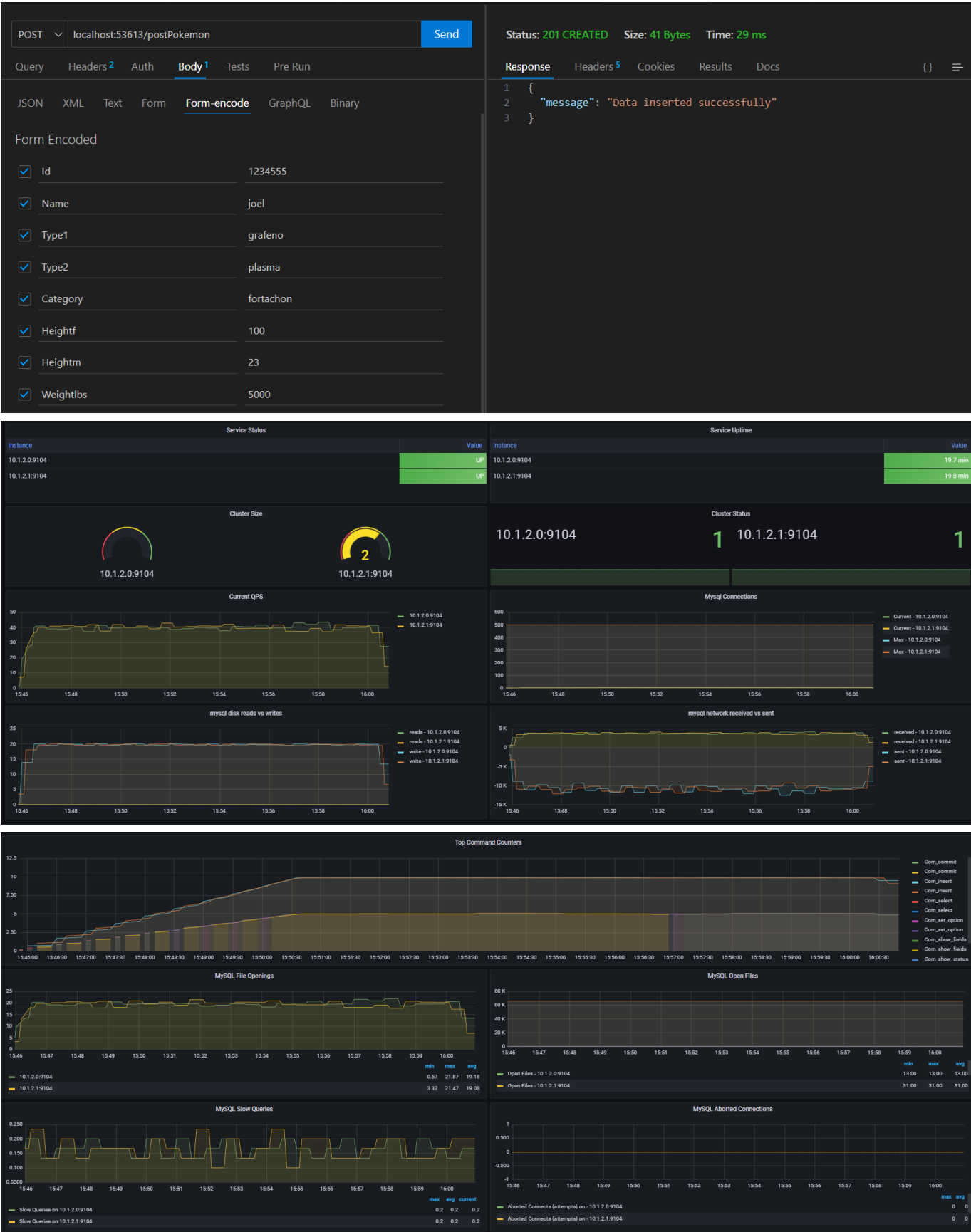
Prueba de Combinaciones



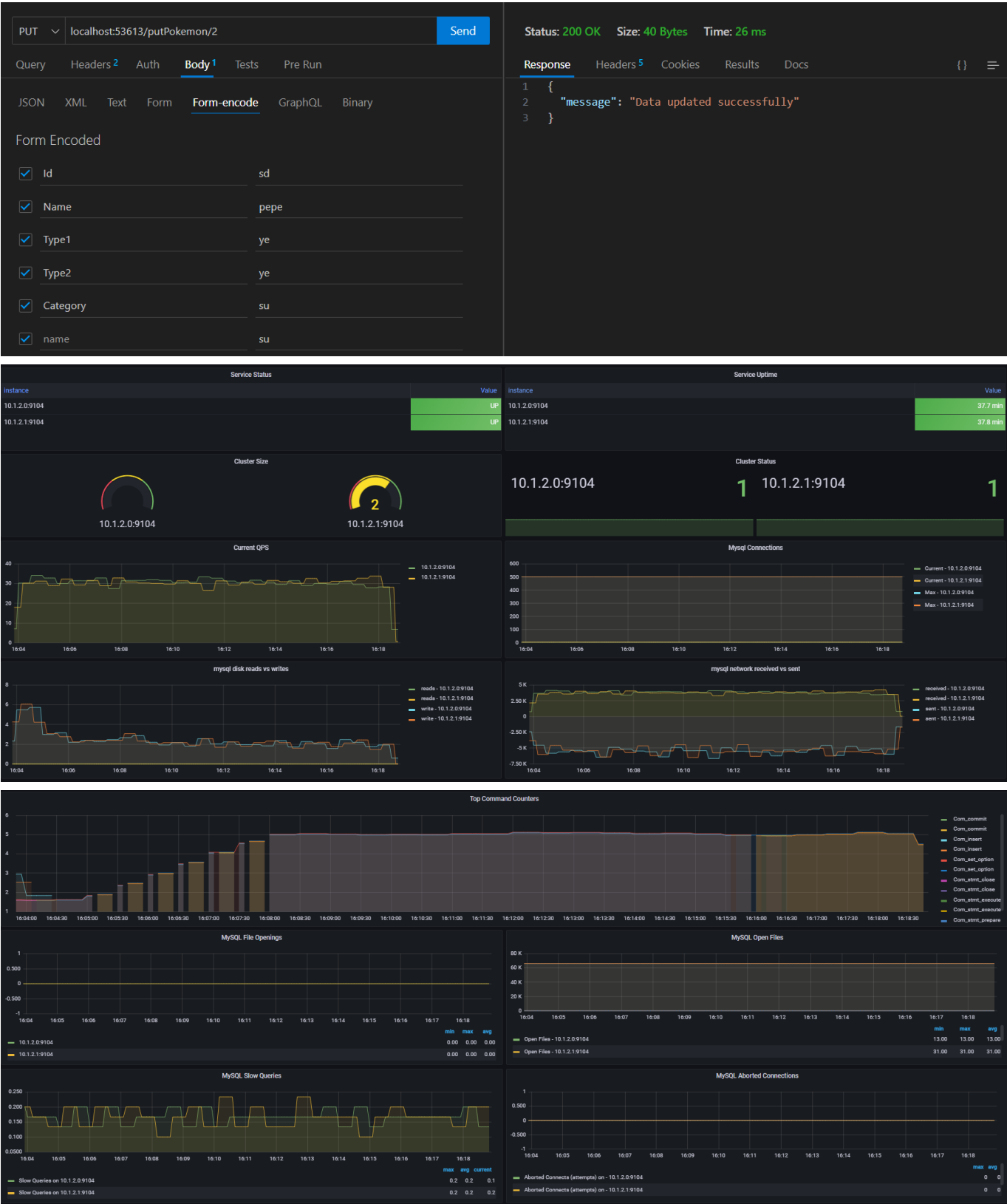
Maria DB Galera

Para la base de datos de Mariadb Galera, con las pruebas realizadas se obtuvieron las siguientes métricas de de Prometheus graficadas en Grafana:

Prueba de Posts



Prueba de Updates



Prueba de Gets

GETlocalhost:53613/getPokemon/2Send

QueryHeadersAuthBodyTestsPre Run

Query Parameters

☐

parameter

value

Status: 200 OKSize: 340 BytesTime: 25 ms

ResponseHeadersCookiesResultsDocs

```
1 {
2   "Attack": "45",
3   "CaptureRate": "190",
4   "Category": "Bat Pokémon",
5   "Defense": "43",
6   "EggSteps": "3840",
7   "ExpGroup": "Medium Fast",
8   "HP": "65",
9   "Heightf": "1'04\"",
10  "Heightm": "0.4",
11  "Id": "527",
12  "Name": "Woobat",
13  "PokemonId": 2,
14  "SpAttack": "55",
15  "SpDefense": "43",
16  "Speed": "72",
17  "Total": "323",
18  "Type1": "Psychic",
19  "Type2": "Flying",
20  "Weightkg": "2.1",
21  "Weightlbs": "4.6"
22 }
```

GETlocalhost:53613/getAllPokemonSend

QueryHeadersAuthBodyTestsPre Run

Query Parameters

☐

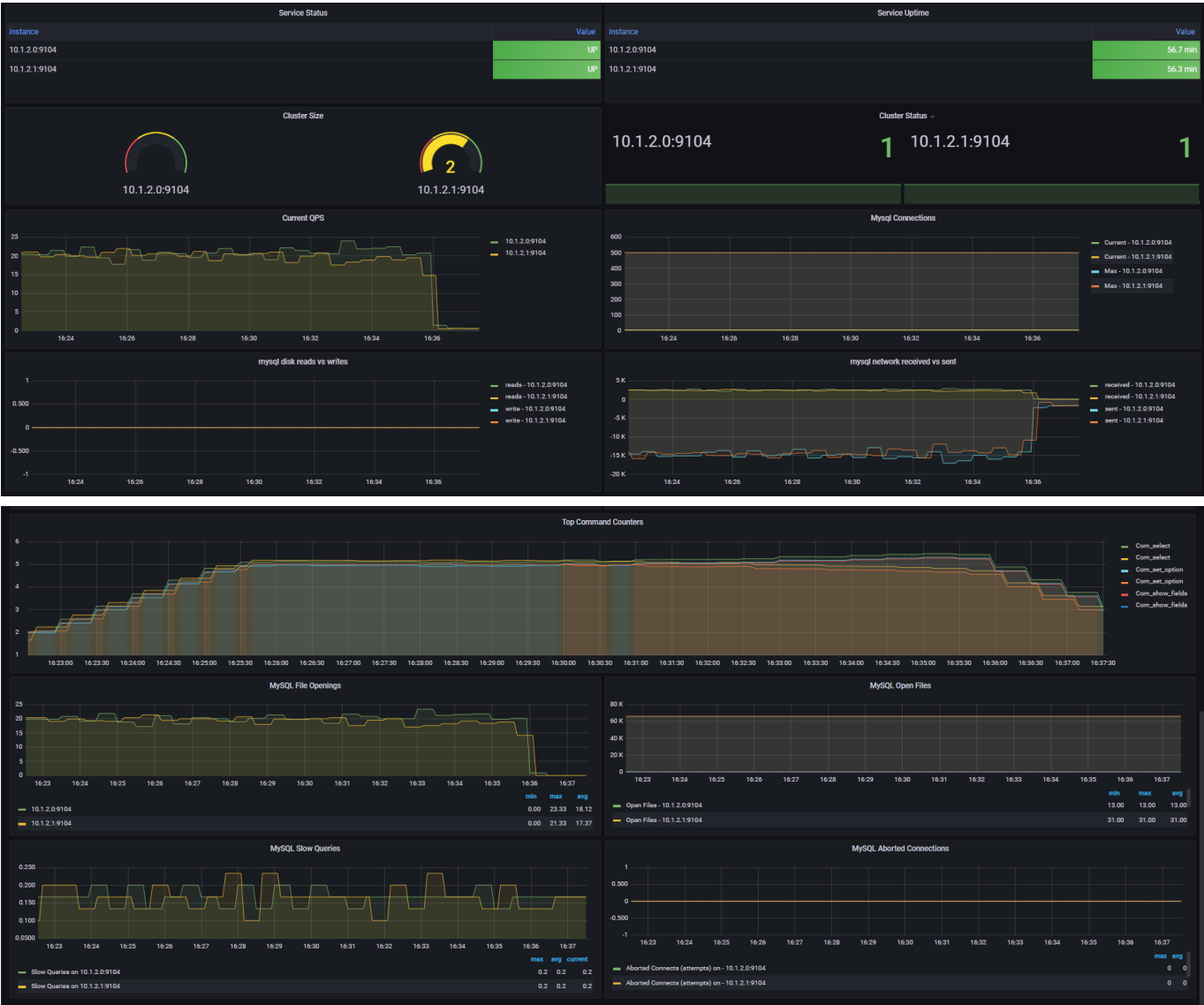
parameter

value

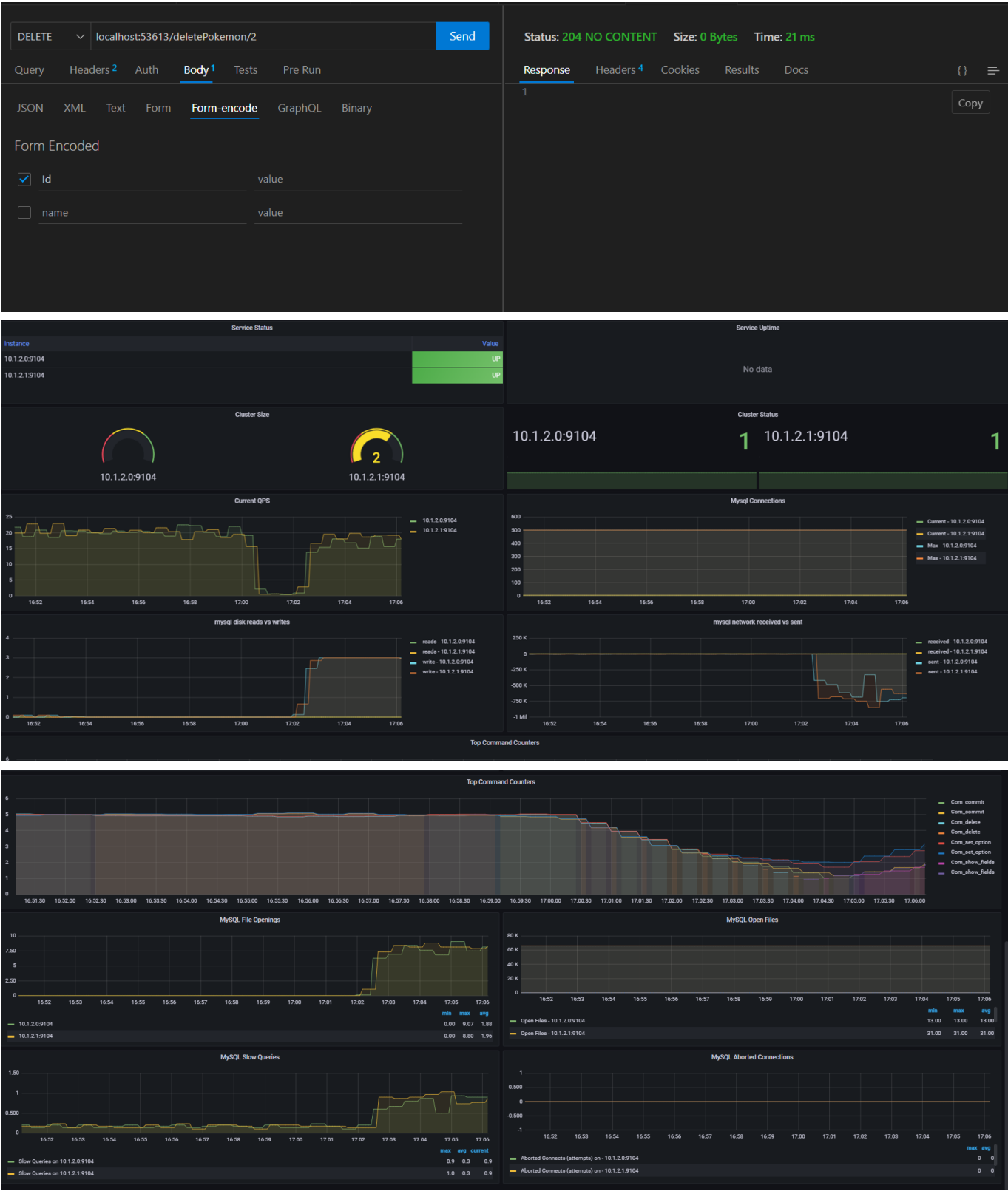
Status: 200 OKSize: 1.41 MBTime: 367 ms

ResponseHeadersCookiesResultsDocs

```
1 [
2   [
3     2,
4     "527",
5     "Woobat",
6     "Psychic",
7     "Flying",
8     "Bat Pokémon",
9     "1'04\"",
10    "0.4",
11    "4.6",
12    "2.1",
13    "190",
14    "3840",
15    "Medium Fast",
16    "323",
17    "65",
18    "45",
19    "43",
20    "55",
21    "43",
22    "72"
23  ],
```



Prueba de Deletes



Service Status

Instance	Value
10.1.2.0:9104	UP
10.1.2.1:9104	UP

Cluster Size

10.1.2.0:9104

210.1.2.1:9104

Current QPS

mysql disk reads vs writes

Service Uptime

No data

Cluster Status

10.1.2.0:9104110.1.2.1:91041

Mysql Connections

mysql network received vs sent

Top Command Counters

Top Command Counters

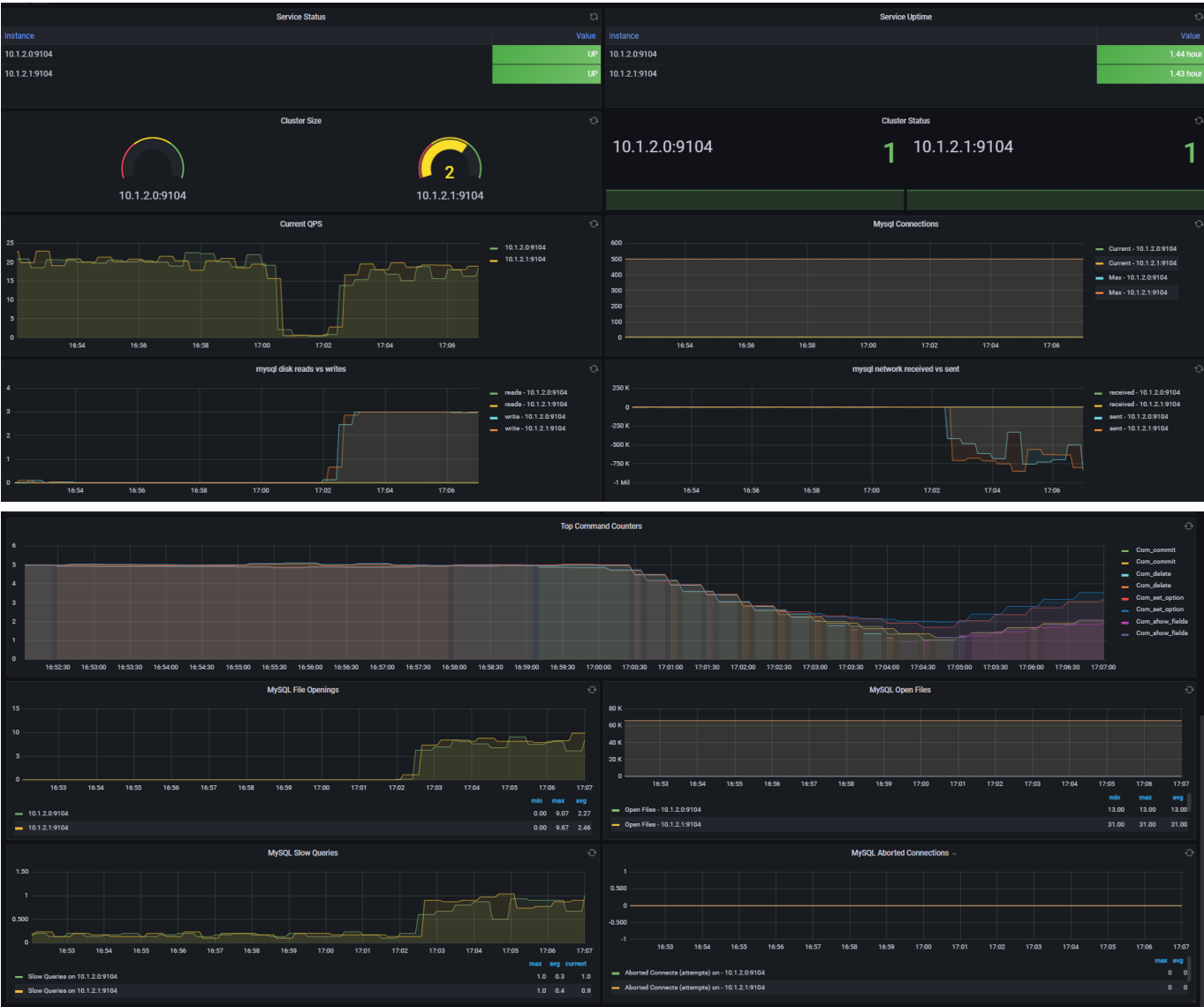
MySQL File Openings

MySQL Open Files

MySQL Slow Queries

MySQL Aborted Connections

Prueba de Combinaciones



Para la aplicación intermediaria se obtuvieron los siguientes datos:

Prueba de Posts



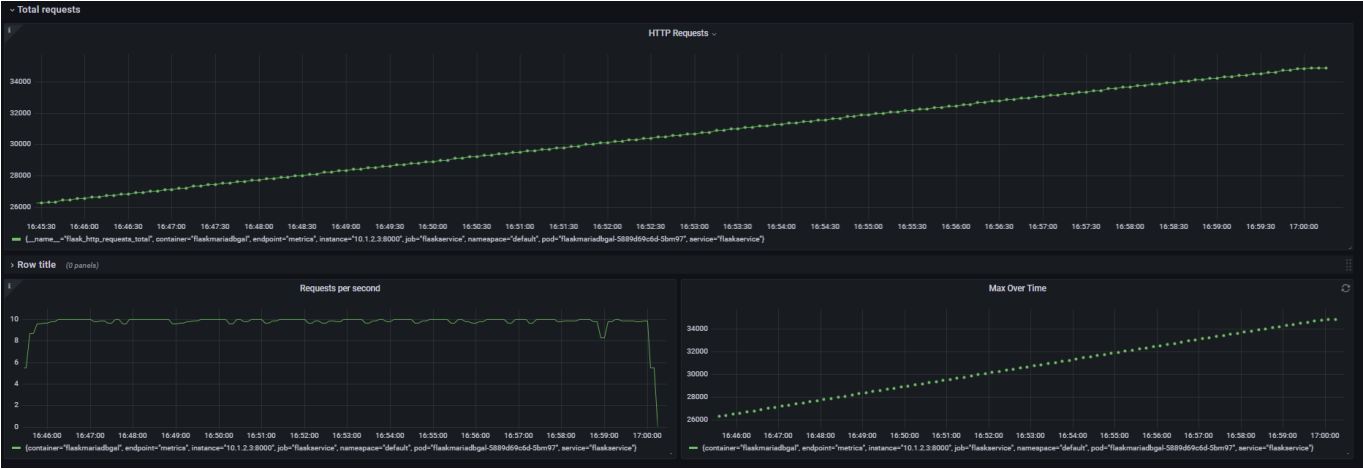
Prueba de Updates



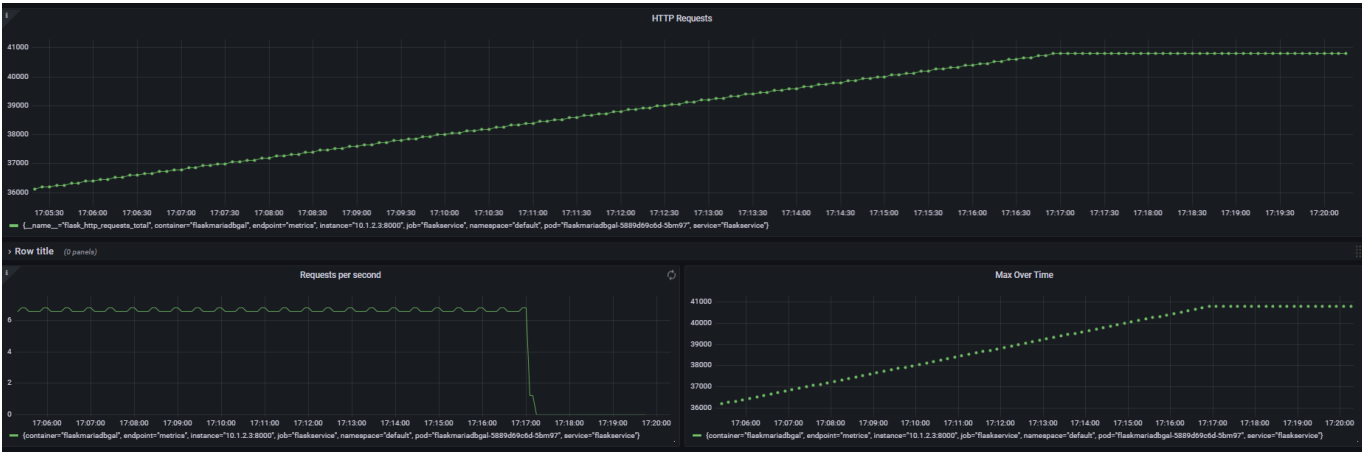
Prueba de Gets



Prueba de Deletes



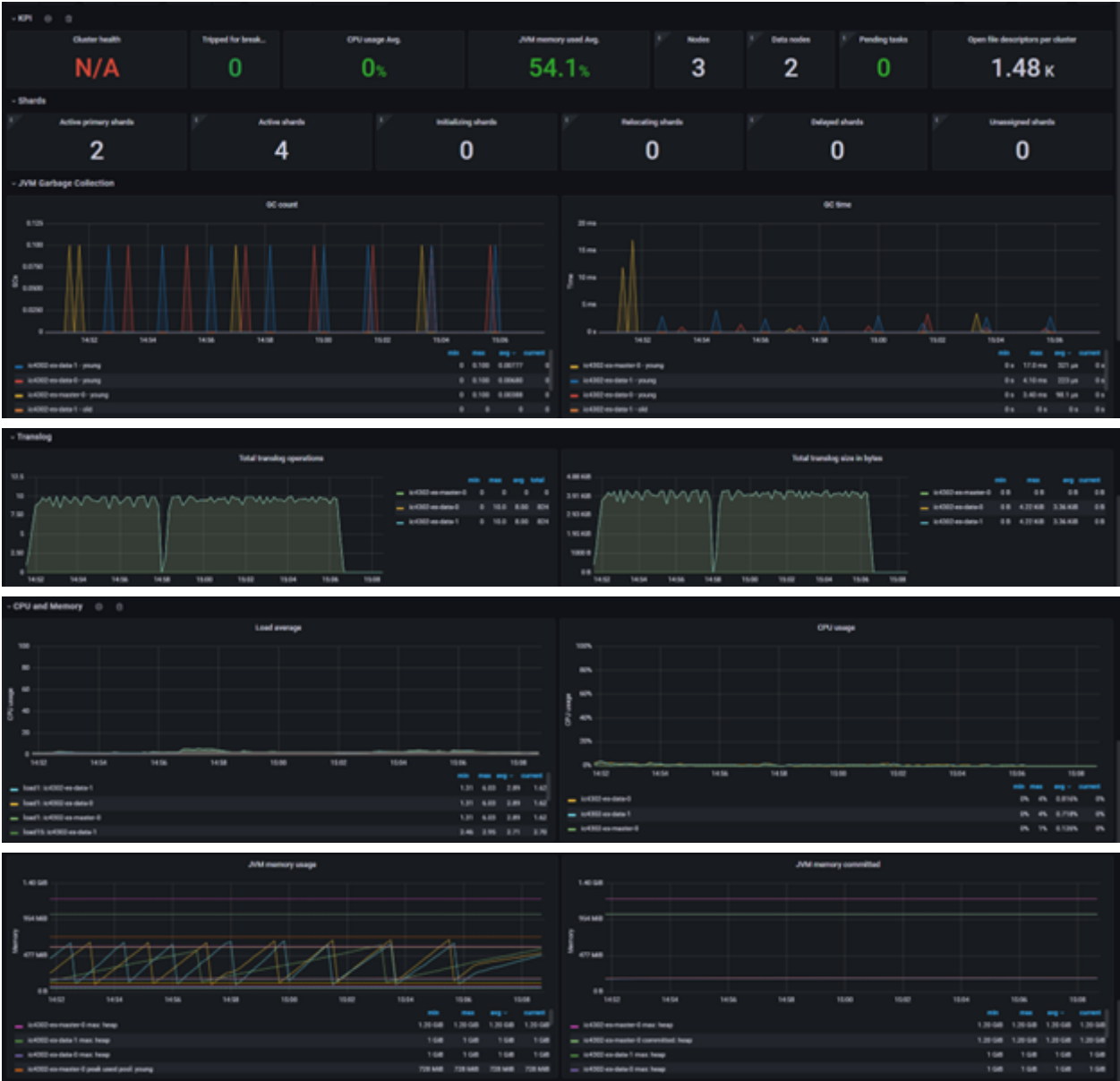
Prueba de Combinaciones



Elasticsearch

Para la base de datos de Elasticsearch, con las pruebas realizadas se obtuvieron las siguientes métricas de Prometheus graficadas en Grafana:

Prueba de Posts





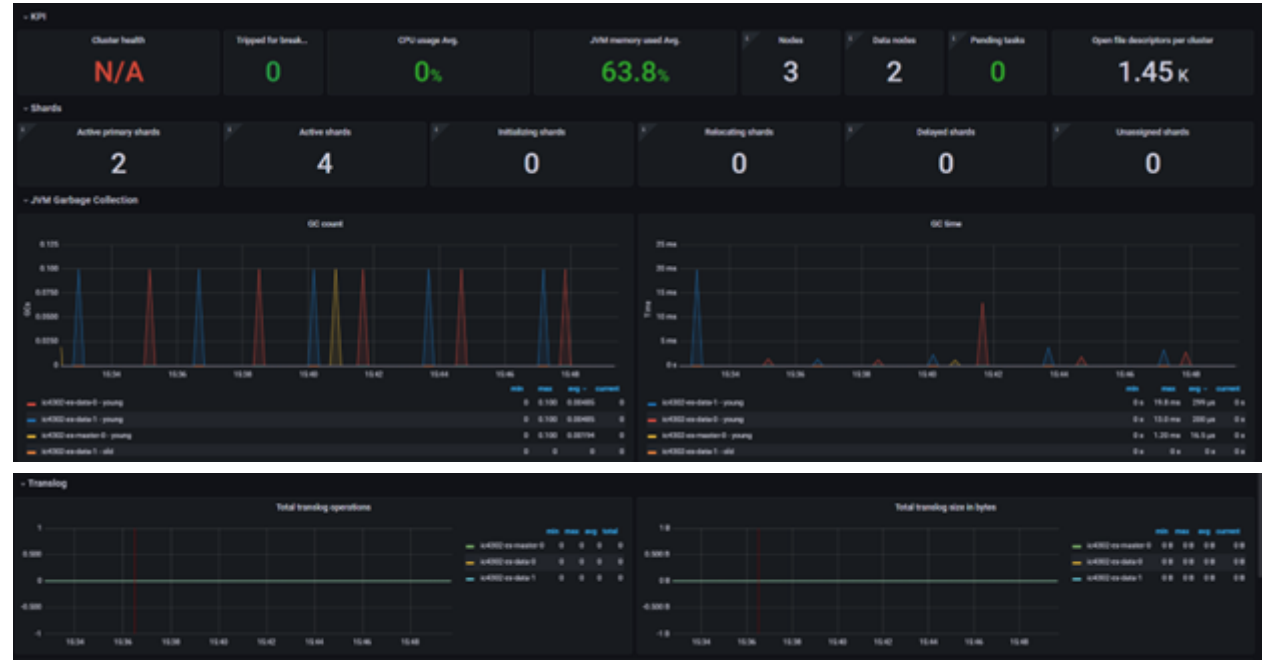


Cantidad actualizada de documentos.

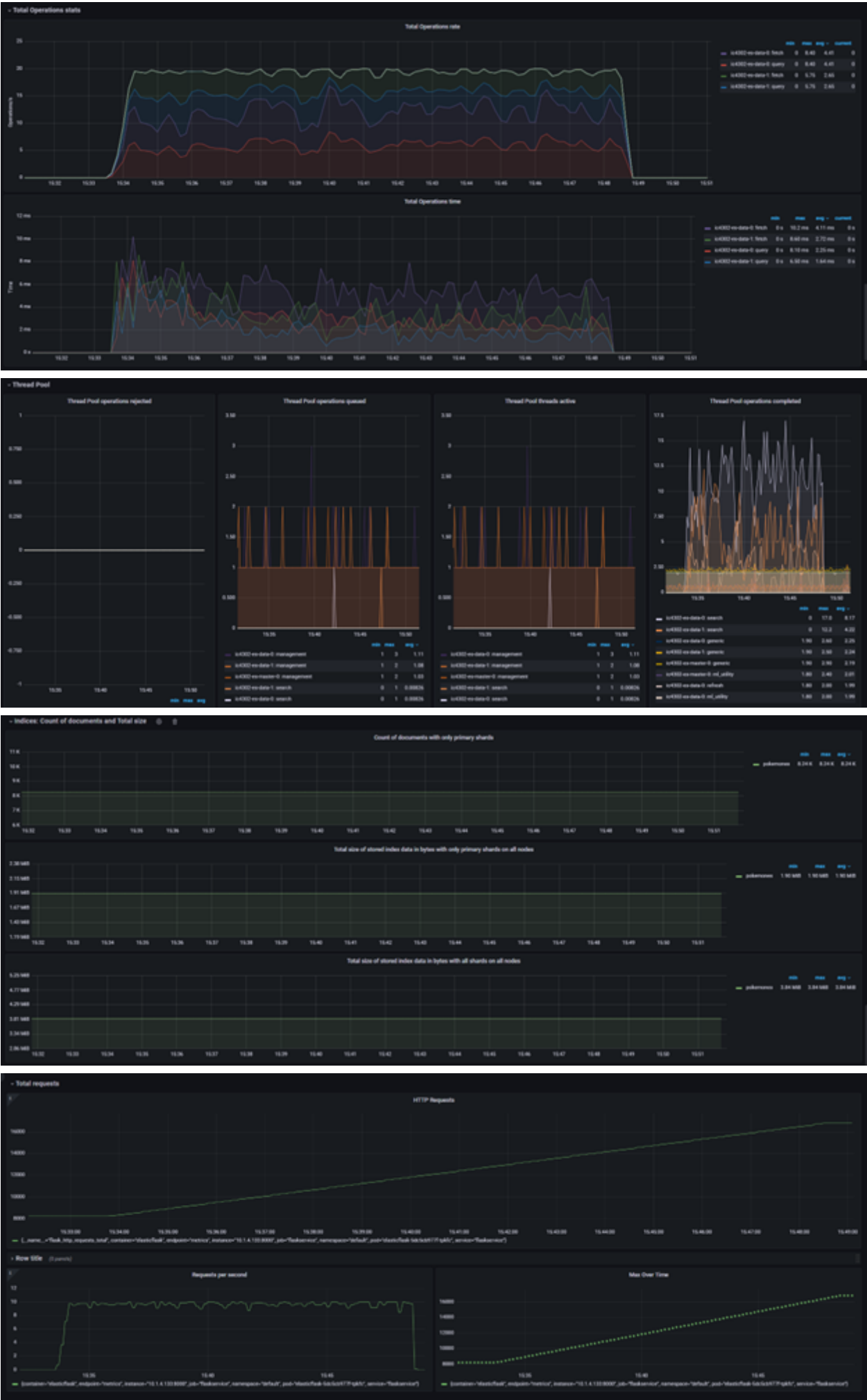


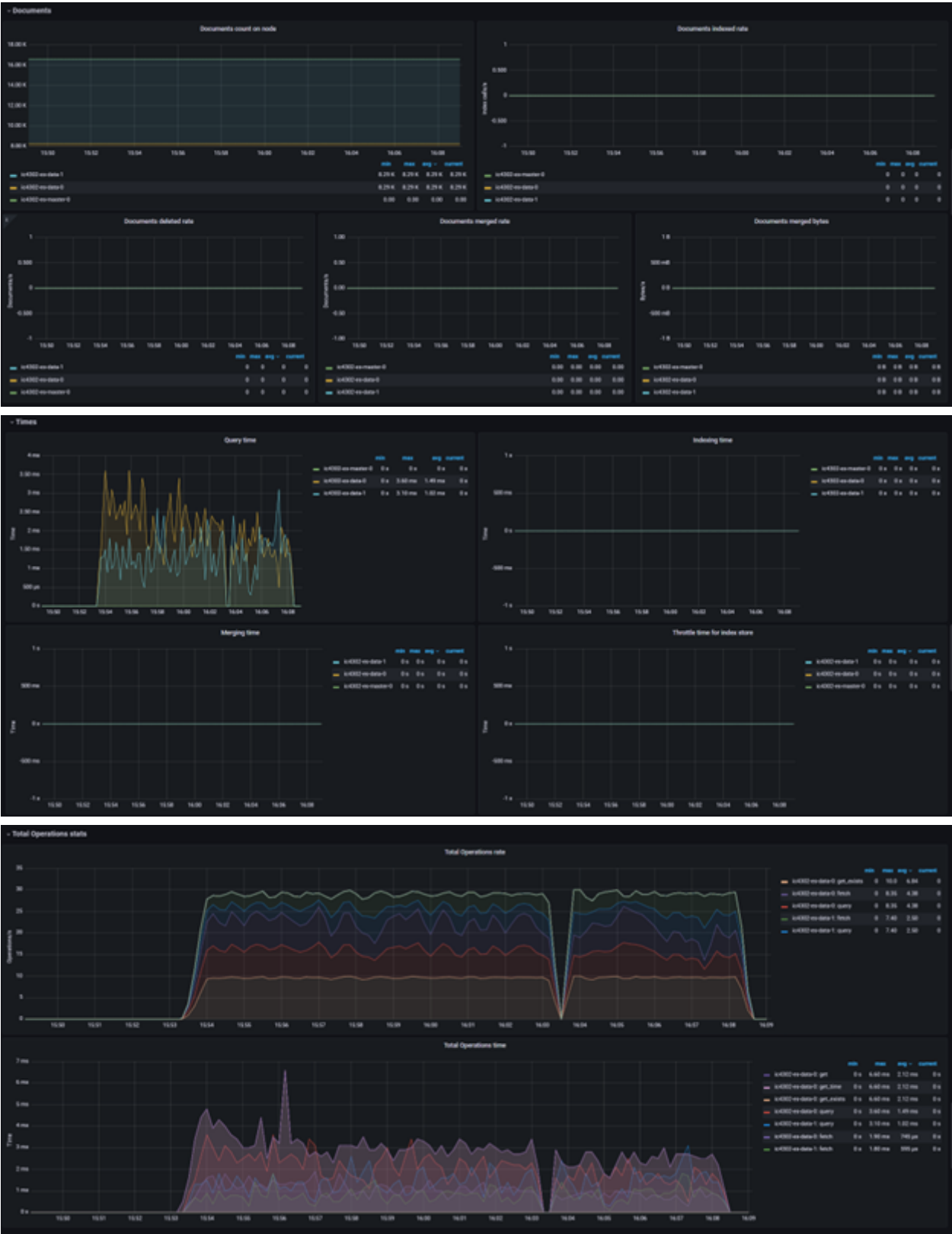


Prueba de Gets



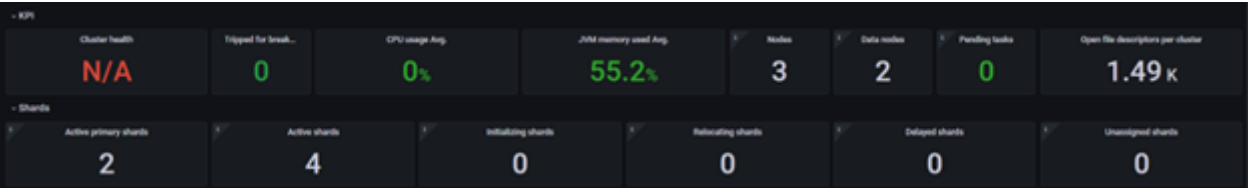








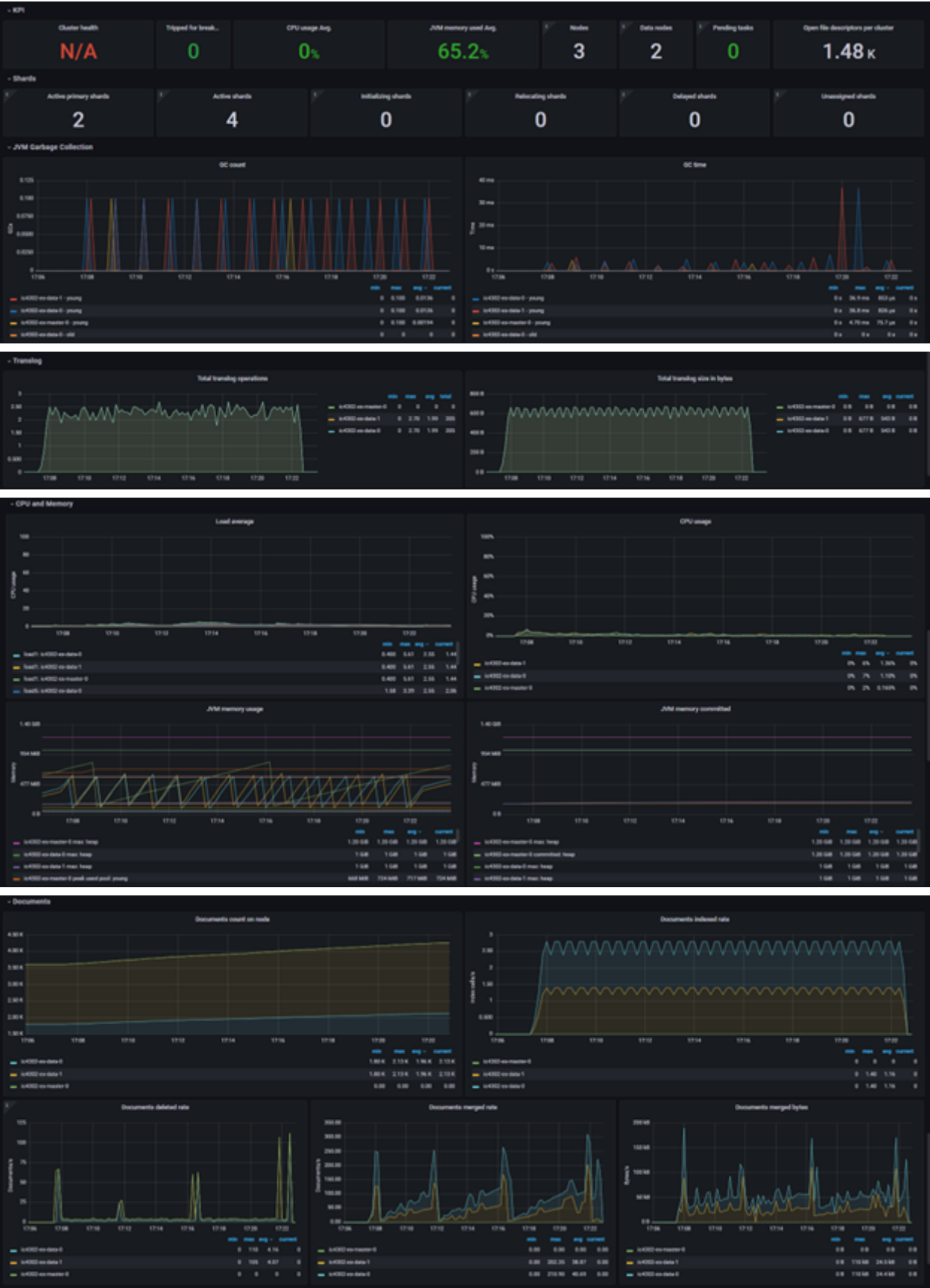
Prueba de Deletes

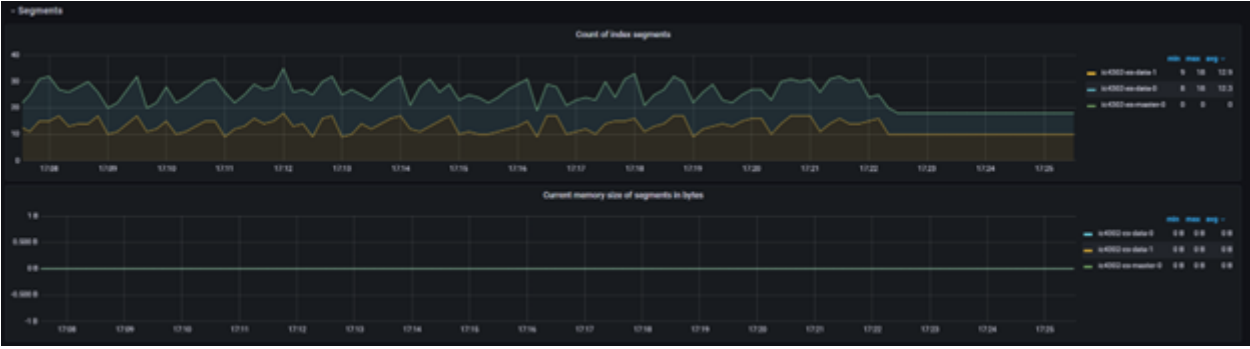
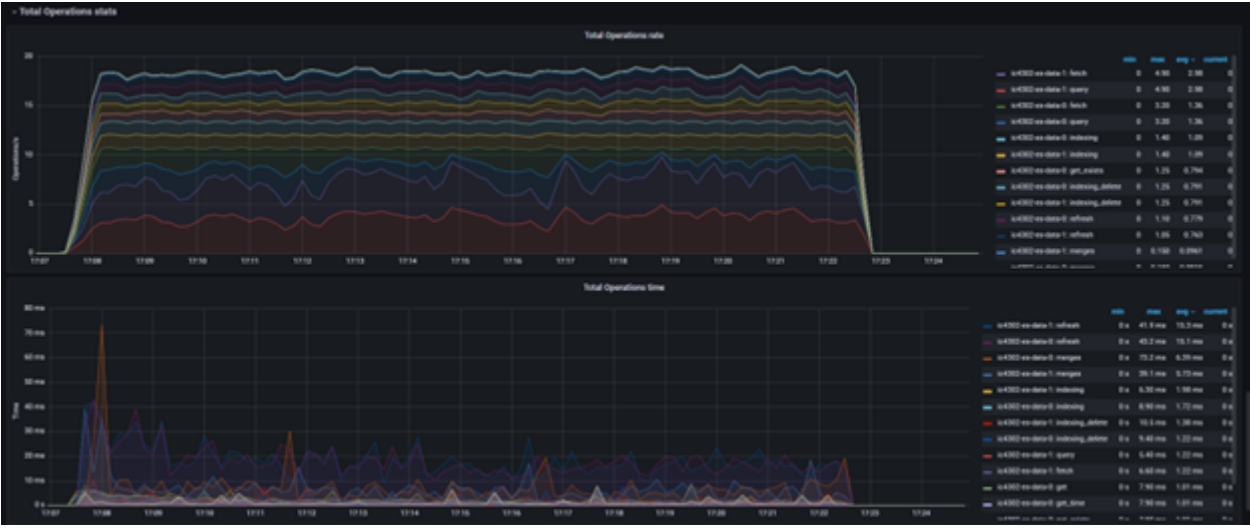














Esta tarea ha sido muy provechosa para aprender nuevas herramientas. En este trabajo, surgieron varios problemas, entonces algunas recomendaciones para resolverlos son:

- Para enviar información de un request de Gatling, esto se puede hacer de varias formas. La primera es utilizando un body. En este, se puede incluir un parámetro con el tipo de información, como RawFileBody, ElFileBody y StringBody. Luego, el body se puede convertir a JSON. Para que el request funcione, el Content-Type header debe concordar con el tipo de datos que se envía. No obstante, cuando intentamos enviar registros a la aplicación intermediaria, daba errores del request. La solución que utilizamos es la segunda forma de enviar información, parámetros en el form. Hicimos que cada campo fuera un parámetro y que luego el feeder los llenara. En la aplicación intermediaria luego se crea un diccionario con el formato de los datos y se inserta la base de datos.
- Cuando un dashboard aparece sin datos, pero Prometheus sí reconoce el service monitor o la base de datos, hay que revisar qué métricas está exponiendo. Puede ser que el dashboard tenga las métricas definidas con diferente nombre, entonces hay que revisar en Prometheus cuáles son las indicadas. Para hacer esto, basta con entrar al servicio y en el apartado de Expression buscar una métrica que tenga la misma información que la del dashboard. Luego, se cambia y se salva el dashboard.
- Para evitar que los cambios en un dashboard se borren, hay que copiar todo el JSON y pegarlo en el archivo en los dashboards del grafana-config.
- Para instalar Gatling correctamente, es necesario tener instalado Maven. Esto es para que se instalen las dependencias. Si por alguna razón no se reconocieran, podemos tratar de darle fix dependencies o fix imports.
- Para que Gatling se comunique con la aplicación intermediaria, es importante hacer un port-forwarding de esta. Esto se puede lograr con Lens, la línea de comandos o por un nodeport.
- Para hacer que una simulación de un escenario de Gatling dure un tiempo específico, hay que poner un ciclo de forever y una duración máxima de lo que se quiere durar. Así siempre va a durar exactamente lo mismo.
- Para hacer una prueba unitaria de un endpoint de Flask, podemos usar una aplicación como Postman o Thunderclient. Estas permiten establecer un request HTTP, con todos sus parámetros y headers. De esta forma, podemos saber cómo debemos mandar la información por Gatling y también si la aplicación intermediaria está agarrando bien la información.
- Es importante revisar los parámetros de cada Helm Chart. Estos permiten configurar las especificaciones de las bases de datos.
- Para facilitar la actualización de las imágenes de Docker, se recomienda crear un Makefile que realice todas las instrucciones de Docker.

Este trabajo también nos ayudó a llegar a varias conclusiones sobre bases de datos:

- Cuando se ejecutaban funciones de inserción, como los posts, updates y deletes se notaba un incremento en las operaciones de escritura. Por otro lado, cuando se hacían gets, se notaba un incremento en las operaciones de lectura.
- Con base en las pruebas de MongoDB, podemos ver cómo funciona la arquitectura con tres réplicas. Cuando observamos los pods de las bases de datos en Lens, vemos que hay otro pod además de las réplicas que es el arbiter. Suponemos que este es el tipo de load balancer, que administra a qué base de datos se realiza una consulta y también realiza el empate en las votaciones de cuál base de datos es la principal. Es como un Witness en MySQL. En la replica set metrics podemos los miembros. El último miembro, el anaranjado, debe ser el primary, ya que este es el que recibe la mayor cantidad operaciones y realiza los inserts. En el Disk IO Utilization, podemos ver que los siempre se mantuvo con

un número muy bajo, sin importar las operaciones. Es posible que esto sea debido a que los datos que insertamos no fueron lo suficientemente pesados para subir el espacio utilizado en la base de datos.

- Sobre las pruebas realizadas en la base de datos de Maria DB hemos obtenido valiosos insights sobre su capacidad para manejar cargas de trabajo variadas y demandantes. Se observó una notable estabilidad durante la ejecución de consultas simples incluso bajo cargas de trabajo intensivas. Además, la capacidad de escalabilidad de MariaDB fue impresionante, demostrando su habilidad para adaptarse a entornos de crecimiento dinámico sin comprometer la integridad de los datos ni la eficiencia operativa.
- Con respecto a las pruebas de rendimiento de PostgreSQL HA se puede notar la capacidad de esta base de datos para gestionar cargas de trabajo intensivas. Las consultas se ejecutaron de manera consistente y eficiente, incluso bajo cargas de trabajo exigentes. La capacidad de configuración de PostgreSQL HA permite a los gestores de bases de datos tener un amplio control sobre las conexiones que se realizan a la base de datos, los recursos que esta puede manejar e incluso que tan estrictas deben de ser las reglas de replicación.
- Adicionalmente, en las operaciones de inserciones al inicio hay un tiempo mucho mayor de inserción y un porcentaje mayor de uso de CPU. Ambas métricas van disminuyendo a medida que progresaba la prueba. A su vez, el uso de memoria empezaba bajo, e iba lentamente subiendo constantemente hacia el final de la prueba. Esto puede representar que al inicio donde no hay tantas páginas guardadas en la memoria, el uso del CPU es mayor ya que hay más probabilidad de Page Faults, Caché Misses, Context Switches, etc. Sin embargo, a medida que más datos se iban cargando a la memoria y por el principio de localidad, cada vez era más fácil para el CPU hacer lecturas de la base de datos, por lo que el uso del CPU podía ir bajando hacia el final.
- El tiempo de borrado es proporcional a la cantidad de registros en las bases SQL. Por lo tanto, se percibe una disminución en lo que dura la base de datos en borrar un registro a medida que la cantidad total de registros va bajando. Esto ocurre debido a que estábamos haciendo las búsquedas con base en un campo que no estaba indexado, por lo que tenía que hacer un recorrido de toda la tabla para encontrarlo. Probablemente, si ese campo hubiera estado indexado, hubiéramos visto un mejor rendimiento en los borrados desde el inicio.
- Elasticsearch usa un sistema de consistencia eventual. Cuando se estaba haciendo las pruebas de inserción, la cantidad de documentos que aparecieron durante el transcurso de la prueba no era tan alta. Estaba en aproximadamente 82 documentos. Esto no era la cantidad real, ya que se realizaron aproximadamente 8000 inserciones. Fue hasta un tiempo después que la cantidad real de documentos se actualizó, por lo que se percibe un incremento súbito en la cantidad de documentos registrados en cada nodo.
- Al tener dos instancias en MariaDB Galera potenció la disponibilidad y redundancia del sistema. Esta configuración nos permite una gestión eficiente de las consultas y garantizó tiempos de respuesta rápidos. En caso de fallos en un nodo, el otro sirvió de respaldo, asegurando la continuidad del servicio. Además, facilitó el mantenimiento sin afectar la disponibilidad.